PipeSD: An Efficient Cloud-Edge Collaborative Pipeline Inference Framework with Speculative Decoding

Yunhe Han [*1] **Yunqi Gao** [*1 2] **Bing Hu** [1] **Mahdi Boloursaz Mashhadi** [3] **Yitong Duan** [4] **Pei Xiao** [3] **Yanfeng Zhang** [2]

Abstract

Speculative decoding can significantly accelerate LLM inference, especially given that its cloud-edge collaborative deployment offers cloud workload offloading, offline robustness, and privacy enhancement. However, existing collaborative inference frameworks with speculative decoding are constrained by (i) sequential token generation and communication with low resource utilization, and (ii) inflexible cloud non-autoregressive verification (NAV) triggering that induces premature verification or costly rollbacks. In this paper, we propose *PipeSD*, an efficient cloud-edge collaborative pipeline inference framework with speculative decoding. PipeSD overlaps token generation and communication by a token-batch pipeline scheduling mechanism optimized by dynamic programming, and improves verification flexibility through a dual-threshold NAV triggering mechanism with a lightweight Bayesian optimization autotuner. We implement PipeSD using llama-cpp-python, PyTorch, and FastAPI, and evaluate it on a real-world cloud-edge testbed with two draft-target model pairs across four scenarios. Results show that PipeSD consistently outperforms state-of-the-art baselines, achieving 1.16×–2.16× speedup and reducing energy consumption by 14.3%–25.3%. Our code is available at `https://github.com/Ghanyunhe/PipeSD`.

1. Introduction

Generative Large Language Models (LLMs) have achieved remarkable success in tasks like dialogue, content creation, and coding (Holmes et al., 2024; Grattafiori et al., 2024; OpenAI et al., 2024). They are predominantly built on a decoder-only transformer architecture, which employs autoregressive generation to produce output sequences, i.e., generating one token at a time (Brown et al., 2020; Touvron et al., 2023a; Dao et al., 2022; Vaswani et al., 2017). However, the inherent sequential dependency of autoregressive generation makes the inference process a significant latency bottleneck. Speculative decoding has emerged as an efficient approach to accelerate LLM inference without degrading output quality (Leviathan et al., 2023; Chen et al., 2023). The core idea is to employ a smaller, faster "draft" model to speculate multiple draft tokens, which are subsequently verified by a larger "target" model in a single forward pass, thereby reducing inference latency while exactly preserving the output distribution of the target model.

Existing studies have deployed speculative decoding in the cloud (Cai et al., 2024; Li et al., 2025b; Zhao et al., 2024) or on edge devices (Xu et al., 2025), both of which exhibit inherent limitations. Cloud-based deployment leverages powerful hardware and diverse optimization frameworks (Kwon et al., 2023; NVIDIA, 2025; Zheng et al., 2024), and enables high-speed execution of large-scale models. However, it depends on continuous network connectivity and raises data privacy concerns (Zhan et al., 2025). In addition, edge deployment supports offline inference and enhanced data privacy (Xu et al., 2025), but is constrained by limited computational resources and memory, hindering the efficient deployment of LLMs.

Compared to the two deployment modes mentioned above, the "draft-and-verify" architecture of speculative decoding is inherently more suitable for cloud-edge collaborative inference mode, which leverages the computing power of both cloud and edge, along with communication between them, to complete inference tasks (Li et al., 2025a; Tian et al., 2024). Specifically, the draft model is deployed at the edge for rapid autoregressive generation, while the target model resides in the cloud for non-autoregressive verifica-

[*]Equal contribution [1]School of Information Science and Electronic Engineering, Zhejiang University, Hangzhou, China [2]School of Computer Science and Engineering, Northeastern University, Shenyang, China [3]5GIC & 6GIC, Institute for Communication Systems (ICS), University of Surrey, Guildford, UK [4]Zhongguancun Institute of Artificial Intelligence, Beijing, China. Correspondence to: Bing Hu <binghu@zju.edu.cn>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

PipeSD：基于推测解码的高效边云协同流水线推理框架

Yunhe Han [*1] **Yunqi Gao** [*1 2] **Bing Hu** [1] **Mahdi Boloursaz Mashhadi** [3] **Yitong Duan** [4] **Pei Xiao** [3] **Yanfeng Zhang** [2]

摘要

推测解码能够显著加速 LLM 推理，其边云协同部署尤其能够卸载云端工作负载、支持离线运行并增强隐私。然而，现有采用推测解码的协同推理框架受限于：(i) token 生成与通信串行执行，导致资源利用率低；(ii) 云端非自回归验证（NAV）的触发方式缺乏灵活性，容易过早验证或产生代价高昂的回滚。本文提出 *PipeSD*，一种基于推测解码的高效边云协同流水线推理框架。PipeSD 通过动态规划优化的 token 批次流水线调度，使 token 生成与通信重叠；并结合轻量级贝叶斯优化自动调优器，利用双阈值 NAV 触发机制提高验证灵活性。我们使用 llama-cpp-python 、 PyTorch 和 FastAPI 实现 PipeSD，并在真实边云测试平台上，使用两组草稿与目标模型组合，在四种场景下开展评测。结果表明，PipeSD 持续优于现有先进基线，获得 1.16×-2.16× 的加速，并降低 14.3%-25.3% 的能耗。代码地址为 `https://github.com/Ghanyunhe/PipeSD`。

1. 引言

生成式大语言模型（ LLMs ）在对话、内容创作和编码等任务中取得了显著成功（ Holmes 等人，2024 ；Grattafiori 等人，2024 ； OpenAI 等人，2024)。它们主要基于仅解码器的 Transformer 架构，使用自回归生成来产生输出序列，即一次生成一个 token （ Brown 等人，2020 ； Touvron 等人，2023a ； Dao 等人 , 2022; Vaswani 等人 ,2017)。然而，自回归生成的固有顺序依赖使推理过程成为显著的时延瓶颈。推测解码已成为一种高效的方法，用于在不降低输出质量的情况下加速大语言模型推理（ Leviathan 等人，2023 ； Chen 等人，2023 ）。其核心思想是使用一个更小、更快的 " 草稿 " 模型来预测多个草稿 token，然后由更大的 " 目标 " 模型在单次前向传递中进行验证，从而在完全保留目标模型输出分布的同时减少推理时延。

现有研究已经在云端部署了推测解码 (Cai et al., 2024; Li et al., 2025b;Zhao et al., 2024) 或在边缘设备上 (Xu etal., 2025 ），两者都存在固有的局限性。基于云的部署利用了强大的硬件和多样的优化框架（ Kwonet al., 2023;NVIDIA, 2025; Zheng etal., 2024 ），并实现大规模模型的高速执行。然而，它依赖于持续的网络连接，并引发数据隐私问题（ Zhan et al.,2025 ）。此外，边缘部署支持离线推理和增强的数据隐私（ Xu et al., 2025 ），但受限于有限的计算资源和内存，阻碍了大语言模型的高效部署。

与上述两种部署模式相比，推测解码的 " 草稿生成与验证 " 架构天然更适合边云协同推理：它结合云端与边缘的计算能力及两侧通信来完成推理任务（ Li et al., 2025a; Tianet al.,2024 ）。具体而言，草稿模型部署在边缘，负责快速自回归生成；目标模型部署在云端，负责非自回归验证

[*] 同等贡献。[1] 浙江大学信息与电子工程学院，中国杭州。[2] 东北大学计算机科学与工程学院，中国沈阳。[3] 萨里大学通信系统研究所（ ICS ），5GIC 与 6GIC，英国吉尔福德。[4] 中关村人工智能研究院，中国北京。通信作者：胡兵 <binghu@zju.edu.cn>。

第 43rd 届国际机器学习会议论文集，韩国首尔。PMLR 306，2026。 版权 2026 归作者所有。

tion (NAV). This mode not only leverages edge resources to reduce cloud workloads but also provides an adaptive execution strategy, e.g., under unstable network conditions or for privacy-sensitive tasks, the edge device can autonomously switch to local inference. Some pioneering work has started exploring the potential of cloud-edge collaborative inference frameworks with speculative decoding, such as HSL (Hao et al., 2024), HAT (Xie et al., 2025), and SpecEdge (Park et al., 2025).

However, existing frameworks face two main limitations: (1) *Sequential token generation and communication.* Traditional frameworks execute token generation, communication, and cloud NAV sequentially, forcing the cloud to wait for the edge to generate and upload all draft tokens and forcing the edge to wait for NAV feedback, thereby underutilizing bandwidth and computing power. (2) *Inflexible NAV triggering mechanism.* Existing frameworks either adopt a fixed draft length for NAV, ignoring task complexity, or rely on single confidence-based triggering conditions, which can delay error detection or lead to excessive speculation.

To address the above limitations, we propose *PipeSD*, an efficient cloud-edge collaborative pipeline inference framework with speculative decoding. We make the following main technical contributions: (1) PipeSD introduces a token-batch pipeline scheduling mechanism that overlaps draft generation and communication to maximize resource utilization and minimize inference latency, which mathematically formulates pipeline scheduling as an optimization problem and obtains an optimal solution by dynamic programming (DP). (2) PipeSD adopts a dual-threshold NAV triggering mechanism to improve verification flexibility, by considering the confidence of both the overall sequence and individual tokens (token confidence is the probability assigned by the draft model to that token and sequence confidence is the product of the confidences of all tokens). It integrates a lightweight Bayesian optimization (BO) autotuner for automatic threshold adaptation. (3) We implement PipeSD using llama-cpp-python (Betlen, Andrei, 2023), PyTorch (Paszke et al., 2019), and FastAPI (FastAPI Developers, 2018), and evaluate it on real-world cloud-edge testbeds. Experiments on two draft-target model pairs across four scenarios demonstrate that PipeSD consistently outperforms state-of-the-art baselines, including HSL and EdgeLLM, achieving 1.16×–2.16× speedup and reducing energy consumption by 14.3%–25.3%.

2. Background and Challenges

2.1. Decoder-based LLMs and Autoregressive Inference

Generative LLMs predominantly adopt the decoder-only transformer architecture (Brown et al., 2020; Touvron et al., 2023a; Dao et al., 2022). This design employs a stack of

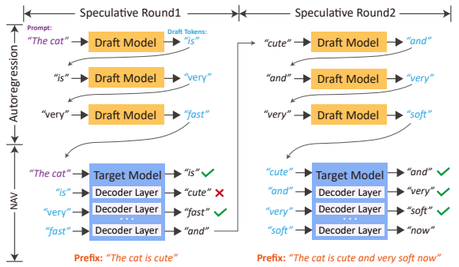


Figure 1. Illustration of the speculative decoding process.

decoder layers, each utilizing masked self-attention and feed-forward networks to process input sequences (Sheng et al., 2023). Decoder-only LLMs typically adopt autoregressive generation, which produces output sequence one token at a time, and each newly generated token is appended to the input sequence to predict the subsequent one (Vaswani et al., 2017; Radford et al., 2019). This inherent sequential dependency, however, makes the inference process a significant latency bottleneck.

2.2. Accelerating Inference with Speculative Decoding

Figure 1 illustrates an example of speculative decoding (Leviathan et al., 2023; Chen et al., 2023), which accelerates the inference process by using a draft model to predict a draft sequence (that consists of multiple draft tokens), and validating it in a single forward pass by the target model. We define a *speculative round* including two steps: First, the draft model autoregressively generates a sequence of draft tokens. Second, all draft tokens are sent to the target model for a single NAV. If the draft tokens match those the target model would generate, they are accepted. Otherwise, the target model corrects the first mismatched token, and the draft tokens before that token are accepted. The accepted and corrected tokens are appended to the output sequence and serve as the prefix for subsequent inference. During an inference task, speculative rounds repeat until the complete output accepted token sequence is produced. This method leverages the high probability that many tokens can be correctly predicted by a draft model on common tasks, thus breaking the strict autoregressive bottleneck of target model and significantly reducing end-to-end inference latency while preserving the output quality.

2.3. Cloud-Edge Collaborative Inference with Speculative Decoding

Speculative decoding is naturally well-suited for a cloud-edge collaborative inference architecture (Hao et al., 2024; Xie et al., 2025; Park et al., 2025), where the lightweight draft model is deployed on a resource-constrained edge de-

（NAV）。这种模式不仅利用边缘资源来减轻云端工作负载，还提供了一种自适应执行策略，例如，在网络不稳定的情况下或对于隐私敏感任务，边缘设备可以自主切换到本地推理。一些开创性工作已经开始探索具有推测解码的边云协同推理框架的潜力，例如 HSL（Zhao 等，，2024）、HAT（Xie 等，，2025）以及 SpecEdge（Park 等，，2025）。

然而，现有框架面临两个主要局限性：(1) 顺序 token 生成和通信。传统框架按顺序执行 token 生成、通信和云 NAV，迫使云端等待边缘生成并上传所有草稿 token，同时迫使边缘等待 NAV 反馈，从而未充分利用带宽和计算能力。(2) 缺乏灵活性的 NAV 触发机制。现有框架要么采用固定草稿长度的 NAV，忽略任务复杂性，要么依赖单一基于置信度的触发条件，这可能导致错误检测时延或产生过度推测。

为了应对上述限制，我们提出了 *PipeSD*，一个高效的边云协同流水线推理框架，带有推测解码。我们的主要技术贡献如下：(1) PipeSD 引入了一种 token 批次流水线调度机制，该机制可以重叠草稿生成和通信，以最大化资源利用率并最小化推理时延，并将流水线调度在数学上表述为一个优化问题，通过动态规划（DP）获得最优解。(2) PipeSD 采用双阈值 NAV 触发机制以提高验证灵活性，通过考虑整体序列和单个 token 的置信度（token 置信度是草稿模型对该 token 分配的概率，序列置信度是所有 token 置信度的乘积）。它集成了轻量级贝叶斯优化（BO）自动调优器用于 自动阈值自适应。（3）我们使用 llama-cpp-python（Betlen,Andrei, 2023）、PyTorch（Paszke 等人 ,2019）和 FastAPI（FastAPI 开发者 , 2018）实现 PipeSD，并在真实的云边缘测试平台上进行评估。在两个草稿 - 目标模型对、四种场景下的实验表明，PipeSD 一直优于最先进的基线方法，包括 HSL 和 EdgeLLM，实现了 1.16×–2.16× 的加速，并将能耗降低了 14.3%–25.3%。

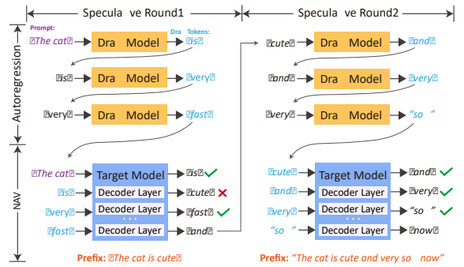


图 1. 推测解码过程的示意图。

2. 背景与挑战

2.1. 基于解码器的大语言模型和自回归推理

生成式 LLM 主要采用仅解码器的 Transformer 架构（Brownet al., 2020;Touvron et al.,2023a; Dao et al., 2022）。该架构由多层堆叠的解码器层，每层使用掩码自注意力和前馈网络来处理输入序列（Sheng 等人 ,2023）。仅解码器的大语言模型通常采用自回归生成，这种方法一次生成一个输出 token，每生成一个新 token 就将其附加到输入序列中以预测下一个 token（Vaswani 等人 , 2017；Radford 等人 , 2019）。然而，这种固有的顺序依赖使推理过程成为一个显著的时延瓶颈。

2.2. 使用推测解码加速推理

图 1 展示了推测解码的一个示例（Leviathan 等人。，2023; Chen 等人。，2023），通过使用草稿模型预测由多个草稿 token 组成的草稿序列，并通过目标模型一次前向验证来加速推理过程。我们将推测轮 包含两个步骤：首先，草稿模型自回归生成一系列草稿 token。其次，所有草稿 token 都发送给目标模型，用于单一 NAV。如果草稿 token 与目标模型生成的符号匹配，则被接受。否则，目标模型会纠正第一个不匹配的 token 并接受该 token 之前的草稿 token。被接受和修正的 token 会被添加到输出序列中，并作为后续推理的前缀。在推理任务中，推测轮次会重复进行，直到生成完整的被接受输出 token 序列为止。这种方法利用了草稿模型在常见任务中可以正确预测许多 token 的高概率，从而打破了目标模型严格的自回归瓶颈，并在保持输出质量的同时显著减少端到端的推理时延。

2.3. 基于推测解码的边云协同推理

推测解码天然适合用于边云协同推理架构（Hao 等人 , 2024； Xie 等人 ,2025； Park 等人 ,2025），其中轻量级草稿模型部署在资源受限的边缘设备上，例如

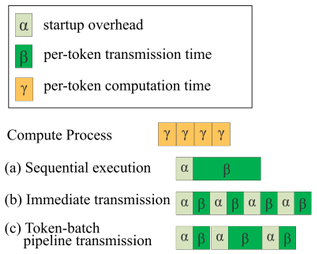


Figure 2. Comparison of transmission strategies.

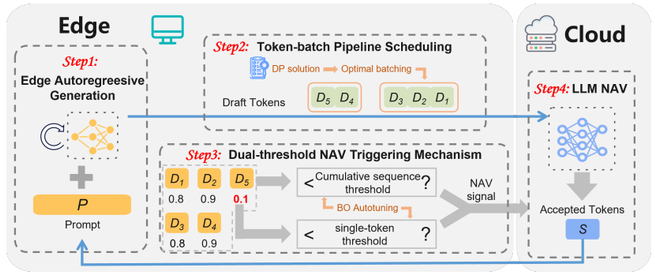


Figure 3. Workflow of PipeSD within one speculative round.

vice (e.g., a smartphone), and the large target model resides in the cloud. In a speculative round of traditional collaborative inference with speculative decoding, the edge device first generates draft tokens autoregressively. Then, the edge transmits them to the cloud for NAV. Finally, the cloud validates the draft tokens using the target model and returns the accepted tokens. This distributed design offers three key advantages: (i) it effectively leverages the computational capability of edge devices to reduce the workload of the cloud; (ii) it enables the local draft model to continue executing lightweight tasks under poor network connectivity; and (iii) edge devices selectively upload inference tasks to the cloud, thereby improving data privacy.

2.4. Performance Bottlenecks

While cloud-edge collaborative inference with speculative decoding holds great promise, existing frameworks are constrained by two factors: (1) **Sequential computation-communication execution pattern.** Existing frameworks typically adopt a compute-first, transmit-later pattern (Hao et al., 2024). Specifically, the edge device generates the entire draft sequence and then transmits it to the cloud as shown in Figure 2(a). This sequential execution results in unnecessary idle time on both the edge and cloud sides, leading to low utilization of bandwidth and computing power; (2) **Inflexible NAV triggering mechanism.** Traditional speculative decoding generates a fixed number of draft tokens in each speculative round, ignoring the varying difficulty of inference tasks (Kim et al., 2023). This tends to result in either premature verification that misses speculation opportunities, or over-speculation that causes large-scale rollbacks. HSL introduces a single-token confidence-based NAV triggering mechanism (Hao et al., 2024), where NAV is triggered if the confidence of any draft token falls below a predefined threshold. However, this mechanism may never activate verification if each token appears moderately confident, causing over-generation. Moreover, EdgeLLM introduces a cumulative sequence confidence metric (Xu et al., 2025), which controls NAV triggering by monitoring the sequence confidence. Since the scheme accumulates the confidence of the entire draft sequence, it may mask tokens with excessively low confidence, leading to delayed error detection. Beyond cloud-edge-specific frameworks, the broader speculative decoding literature has also explored more adaptive NAV triggering strategies. (Zhang et al., 2025) uses the entropy of the draft-tokens as an uncertainty signal to determine when to trigger NAV. (Huang et al., 2025) triggers NAV based on a learned prediction signal. Nevertheless, these methods still rely on a single signal to control verification, which may be insufficient to jointly capture token-level and sequenc-level confidencee.

The above two limitations motivate us to design a cloud-edge speculative decoding framework with high hardware resource utilization and flexible NAV triggering mechanisms to accelerate collaborative inference, which involves three main challenges:

Complex pipeline scheduling considering cloud-edge communication startup overhead. Intuitively, the communication of draft tokens can be overlapped with the autoregression of the draft model to improve resource utilization and to reduce inference latency, i.e., a draft token can be immediately transmitted once it is generated, as illustrated in Figure 2(b). However, the large startup overhead of communication mitigates the gains of this scheme. Therefore, standard pipeline patterns can hardly be applied.

Joint impact of sequence-level and token-level confidence on NAV triggering. Ideally, the triggering of NAV should be determined by the confidence of both entire draft sequence and individual tokens. Therefore, how to incorporate two factors to jointly determine NAV triggering and determine the optimal values of the two factors is challenging.

Design of an adaptive and compatible collaborative inference framework. In cloud-edge collaborative scenarios, the computing power of edge devices and the bandwidth of cloud-edge communication are usually unstable. An adaptive framework that can automatically adjust parameter configurations according to hardware environment changes is crucial. Furthermore, to ensure the compatibility of the designed framework with various cloud inference optimization

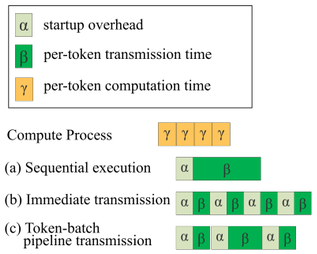


图 2. 传输策略比较。

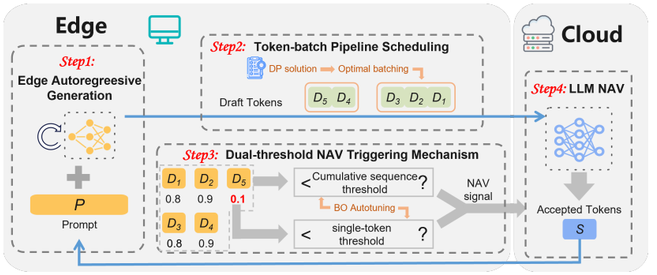


图 3. 在一个推测轮次中 PipeSD 的工作流程。

智能手机，而大型目标模型驻留在云端。在使用推测解码的传统协同推理中，边缘设备首先自回归地生成草稿 token。然后，边缘将它们传输到云端进行 NAV。最后，云端使用目标模型验证草稿 token，并返回被接受的 token。这种分布式设计提供了三个关键优势：（i）它有效利用边缘设备的计算能力以减少云端的工作负载；（ii）它使本地草稿模型能够在网络连接不佳的情况下继续执行轻量级任务；（iii）边缘设备选择性地将推理任务上传到云端，从而提高数据隐私性。

2.4. 性能瓶颈

虽然边云协同推理与推测解码具有巨大的潜力，但现有框架受到两个因素的限制：(1) **顺序计算 - 通信执行模式**。现有框架通常采用先计算、后传输的模式 (Hao 等，，2024)。具体来说，边缘设备生成整个草稿序列，然后将其传输到云端，如图 2(a) 所示。这种顺序执行导致边缘和云端两端出现不必要的空闲时间，从而导致带宽和计算能力的低利用率；(2) **缺乏灵活性的 NAV 触发机制**。传统的推测解码在每次推测时生成固定数量的草稿 token 忽略推理任务难度的不同而采取统一处理（ Kim 等人 ., 2023 ）。这往往导致要么过早验证而错过推测机会，要么过度推测导致大规模回滚。HSL 引入了一种基于单个 token 置信度的 NAV 触发机制（ Hao 等人 ., 2024 ），当任何草稿 token 的置信度低于预设阈值时触发 NAV。然而，如果每个 token 表现出中等置信度，这一机制可能永远不会启动验证，从而导致过度生成。此外，EdgeLLM 引入了累积序列置信度指标（ Xu 等人 ., 2025 ），它通过监控序列置信度来控制 NAV 触发。由于该方案累积了整个草稿序列的置信度，可能会掩盖置信度过低的 token，从而导致错误检测时延。除了云 - 边缘特定框架外，更广泛的推测解码文献也探索了更自适应的 NAV 触发策略。(张等人 ,2025) 使用草稿 token 的熵作为不确定性信号来确定何时触发 NAV。(Huang 等人 ,2025) 基于学习的预测信号触发 NAV。然而，这些方法仍然依赖单一信号来控制验证，这可能不足以同时捕捉 token 级和序列级置信度。

上述两项限制促使我们设计一个云边推测解码框架，该框架具有高硬件资源利用率和灵活的 NAV 触发机制，以加速协作推理，这涉及三个主要挑战：

考虑云边通信启动开销的复杂流水线调度。 直观地，草稿 token 的通信可以与草稿模型的自回归重叠，以提高资源利用率并减少推理时延，即一旦生成草稿 token 就可以立即传输，如图所示 2(b) 然而，通信的大量启动开销抵消了这种方案的收益。因此，标准的流水线模式几乎无法应用。

序列级和 token 级置信度对 NAV 触发的联合影响。理想情况下，NAV 的触发应由整个草稿序列以及单个 token 的置信度共同决定。因此，如何将这两个因素结合起来共同决定 NAV 触发，并确定这两个因素的最佳值是具有挑战性的。

自适应且兼容的协作推理框架设计。在边云协同场景中，边缘设备的计算能力和云 - 边通信的带宽通常是不稳定的。一个能够根据硬件环境变化自动调整参数配置的自适应框架是至关重要的。此外，为了确保所设计的框架与各种云推理优化的兼容性

frameworks, optimization mechanisms should be deployed at the edge side to reduce dependency on the cloud.

In this paper, our main goal is to address the above three challenges by proposing an efficient cloud-edge collaborative pipeline inference framework with speculative decoding called PipeSD. All frequently used notations are summarized in Table A.1 of the Appendix.

3. PipeSD Design

3.1. Overview

Figure 3 illustrates the overall workflow of PipeSD within one speculative round, which includes four steps. Step 1: The edge device autoregressively generates draft tokens using a draft model. Step 2: While generating draft tokens, the edge batches and transmits them to overlap communication with computation (see details in Sec. 3.2), where an optimal batching strategy can be obtained by a DP algorithm (see details in Sec. 4.1). Step 3: The edge continuously evaluates both single-token and sequence confidence to determine whether to trigger NAV, and a lightweight BO autotuner is introduced to dynamically adjust the trigger thresholds (see details in Sec. 3.3). Step 4: Once NAV is requested, the cloud validates the accumulated draft tokens using a target model and returns the accepted tokens to the edge.

Steps 2 and 3 are the core steps of the proposed PipeSD. Step 2 effectively addresses the performance bottlenecks of low utilization of bandwidth and computing power by pipelining token generation and communication. Step 3 significantly enhances the flexibility of NAV triggering by introducing dual-threshold verification to simultaneously consider global and individual confidence.

3.2. Token-Batch Pipeline Scheduling Mechanism

We design an efficient token-batch pipeline scheduling mechanism that overlaps token autoregressive generation and communication, by deciding whether to merge draft tokens into one batch or transmit them immediately. For example, as shown in Figure 2(c), batching tokens 2 and 3 can further reduce total communication latency compared to immediate transmission. Therefore, we build a mathematical model to find the optimal token batching strategy that minimizes the autoregressive generation and communication time of draft tokens in a speculative round.

Given the number of draft tokens N in each speculative round, we parameterize the batching strategy by a strictly increasing boundary sequence

$$\mathbb{B} = (b_1, \ldots, b_K), 1 = b_1 < b_2 < \cdots < b_K \leq N \quad (1)$$

where K is the number of token batches and b_k is the index of the first draft token in the k-th token batch.

Then, the cloud-edge communication time of batch k, denoted by $t_c^{(k)}$, can be modeled as

$$t_c^{(k)} = \begin{cases} \alpha + \beta \cdot (b_{k+1} - b_k) & 1 \leq k < K \\ \alpha + \beta \cdot (N + 1 - b_k) & k = K \end{cases} \quad (2)$$

where α is the startup overhead and β is the per-token transmission time (see Sec. 5.2.4 for empirical validation and Appendix A for the modeling rationale). The autoregressive generation time of batch k, denoted by $t_{ag}^{(k)}$, can be represented as

$$t_{ag}^{(k)} = \begin{cases} \gamma \cdot (b_{k+1} - b_k) & 1 \leq k < K \\ \gamma \cdot (N + 1 - b_k) & k = K \end{cases} \quad (3)$$

where γ is the per-token computing time (γ is assumed approximately constant within the optimization window, see Sec. 5.2.4 for empirical validation).

The autoregressive generation of a batch can start only after the previous batch's generation is complete. Thus the generation start time of batch k, denoted by $\tau_{ag}^{(k)}$, can be represented as:

$$\tau_{ag}^{(k)} = \begin{cases} 0 & k = 1 \\ \tau_{ag}^{(k-1)} + t_{ag}^{(k-1)} & k > 1 \end{cases} \quad (4)$$

Similarly, the communication of a batch can start only after both the previous batch's communication is completed and the current batch's generation is finished. Thus the communication start time of batch k, denoted by $\tau_c^{(k)}$, can be represented as:

$$\tau_c^{(k)} = \begin{cases} \tau_{ag}^{(k)} + t_{ag}^{(k)} & k = 1 \\ \max\left\{\tau_c^{(k-1)} + t_c^{(k-1)},\ \tau_{ag}^{(k)} + t_{ag}^{(k)}\right\} & k > 1 \end{cases} \quad (5)$$

Our goal is to find the optimal $\mathbb{B}$ that minimizes the autoregressive generation and communication time of draft tokens in a speculative round. The objective function and the dependencies can be expressed as follows:

$$\begin{aligned} \min_{\mathbb{B}} \quad & T = \tau_c^{(K)} + t_c^{(K)} - \tau_{ag}^{(1)} \\ \text{s.t.} \quad & Eqs.\ (1) - (5) \end{aligned} \quad (6)$$

This optimization problem can be efficiently solved using a DP algorithm, which we describe in detail in Sec. 4.1.

3.3. Dual-threshold NAV Triggering Mechanism

We present a dual-threshold NAV triggering mechanism by jointly considering cumulative sequence confidence and single-token confidence. It precisely determines when to start the cloud NAV, avoiding both premature verification and excessive rollbacks.

For a draft token D_n, we denote its confidence as $P(D_n)$, i.e., the probability assigned by the draft model. Then, we use C_1 to denote the cumulative sequence confidence, which is the product of the confidence of all draft tokens that have

框架、优化机制应部署在边缘端，以减少对云端的依赖。

本文通过提出一种基于推测解码的高效边云协同流水线推理框架 PipeSD，解决上述三项挑战。常用符号汇总于附录表 A.1。

3. PipeSD 设计

3.1. 概述

图 3 说明了 PipeSD 在一次推测轮次中的整体工作流程，包括四个步骤。步骤 1：边缘设备使用草稿模型自回归生成草稿 token。步骤 2：在生成草稿 token 的同时，边缘设备对它们进行批处理并传输，以实现计算与通信的重叠（详情见 Sec. 3.2），其中可以通过 DP 算法获得最优批处理策略（详情见 Sec. 4.1）。步骤 3：边缘设备持续评估单个 token 和序列的置信度，以确定是否触发 NAV，并引入轻量级 BO 自动调优器动态调整触发阈值（详情见 Sec. 3.3）。步骤 4：一旦请求 NAV，云端使用目标模型验证累积的草稿 token，并将接受的 token 返回到边缘 .

步骤 2 和步骤 3 是所提出的 PipeSD 的核心步骤。步骤 2 通过对 token 生成和通信进行流水线处理，有效地解决了带宽和计算能力利用率低的性能瓶颈。步骤 3 通过引入双阈值验证显著增强了 NAV 触发的灵活性，从而同时考虑全局和个体的置信度。

3.2. token 批次流水线调度机制

我们设计了一种高效的 token 批次流水线调度机制，通过决定是否将草稿 token 合并成一个批次或立即传输，实现 token 自回归生成与通信的重叠。例如，如图 2(c) 所示，将 token 2 和 3 批处理，与立即传输相比，可以进一步减少总通信时延。因此，我们建立了一个数学模型，以找到在推测回合中最小化草稿 token 自回归生成和通信时间的最优 token 批次策略。

鉴于每轮投机中草稿代币 N 的数量，我们通过严格递增的边界序列对批处理策略进行参数化

$$\mathbb{B} = (b_1, \ldots, b_K), 1 = b_1 < b_2 < \cdots < b_K \leq N \quad (1)$$

其中 K 是 token 批次的数量，b_k 是 k-th token 批次中第一个草稿 token 的索引。

然后，批次 k 的云 - 边通信时间，记为 $t_c^{(k)}$，可以建模为

$$t_c^{(k)} = \begin{cases} \alpha + \beta \cdot (b_{k+1} - b_k) & 1 \leq k < K \\ \alpha + \beta \cdot (N + 1 - b_k) & k = K \end{cases} \quad (2)$$

其中 α 是启动开销，β 是每个 token 的传输时间（参见 Sec.5.2.4 以获得实证验证，以及附录 A 解释建模原理）。批次 k 的自回归生成时间，记为 $t_{ag}^{(k)}$，可表示为

$$t_{ag}^{(k)} = \begin{cases} \gamma \cdot (b_{k+1} - b_k) & 1 \leq k < K \\ \gamma \cdot (N + 1 - b_k) & k = K \end{cases} \quad (3)$$

其中 γ 是每个 token 的计算时间（γ 被假定在优化窗口内大致恒定，见 Sec.5.2.4 以获得经验验证）。

一批次的自回归生成只能在上一批次生成完成后开始。因此，批次 k 的生成开始时间，用 $\tau_{ag}^{(k)}$ 表示，可以表示为：

$$\tau_{ag}^{(k)} = \begin{cases} 0 & k = 1 \\ \tau_{ag}^{(k-1)} + t_{ag}^{(k-1)} & k > 1 \end{cases} \quad (4)$$

同样，一批的通信只能在前一批的通信完成且当前批的生成完成之后才能开始。因此，批次 k 的通信开始时间，记作 $\tau_c^{(k)}$，可以表示为：

$$\tau_c^{(k)} = \begin{cases} \tau_{ag}^{(k)} + t_{ag}^{(k)} & k = 1 \\ \max\left\{\tau_c^{(k-1)} + t_c^{(k-1)},\ \tau_{ag}^{(k)} + t_{ag}^{(k)}\right\} & k > 1 \end{cases} \quad (5)$$

我们的目标是找到最优的 $\mathbb{B}$，以最小化自回归生成和推测轮次中草稿 token 的通信时间。目标函数及其依赖关系可以表示如下：

$$\begin{aligned} \min_{\mathbb{B}} \quad & T = \tau_c^{(K)} + t_c^{(K)} - \tau_{ag}^{(1)} \\ \text{s.t.} \quad & Eqs.\ (1) - (5) \end{aligned} \quad (6)$$

这个优化问题可以使用动态规划算法高效解决，我们将在第 4.1. 节中详细描述。

3.3. 双阈值净值触发机制

我们提出了一种双阈值 NAV 触发机制，通过联合考虑累积序列置信度和单个 token 置信度。它能够精确地确定何时启动云端 NAV，避免过早验证和过度回滚。

对于草稿 token D_n，我们将其置信度表示为 $P(D_n)$，即草稿模型分配的概率。然后，我们用 C_1 来表示累积序列置信度，它是所有已出现草稿 token 置信度的乘积。

been computed but not yet verified, and R_1 to denote the cumulative sequence confidence threshold. When the draft model autoregressively generates the n-th draft token D_n, PipeSD updates the tentative cumulative confidence C_1^* by multiplying $P(D_n)$ with the current C_1. If $C_1^* \leq R_1$, a cloud NAV is triggered, and C_1 is reset to 1. In addition, we use R_2 to denote the single-token confidence threshold. If $P(D_n) \leq R_2$, a cloud NAV is also triggered. Therefore, the proposed dual-threshold NAV triggering mechanism jointly considers both the confidence of the overall draft sequence and that of individual tokens, effectively improving the flexibility of NAV triggering.

It is worth noting that the threshold pair (R_1, R_2) is closely related to the difficulty of the inference task, and has a significant impact on the speed of collaborative inference. Unfortunately, it is non-trivial to explicitly model the average generation time per accepted token (TPT) as a function of (R_1, R_2). To ensure the adaptivity of PipeSD, we design a lightweight *BO autotuner* to automatically adjust (R_1, R_2).

The BO autotuner aims to identify promising parameters of an unknown objective function using as few samples as possible. In our setting, the objective is to minimize the average TPT. The BO autotuner samples different triplets (R_1, R_2, TPT), and continuously suggests the next threshold pair (R_1, R_2) to obtain a new objective value. The average TPT corresponding to each (R_1, R_2) is measured by averaging the generation time of multiple accepted tokens. In other words, the BO autotuner efficiently predicts near-optimal thresholds after collecting only a small number of samples. In practical inference, with only 16 samples, the BO autotuner is able to return a near-optimal threshold pair. See Sec. 5.2.3 and Appendix C for more details on the performance evaluation and parameter settings of the BO autotuner.

Notably, under the dual-threshold triggering mechanism in PipeSD, the draft length N of each speculative round is dynamically changing and implicitly determined by the confidence of the generated tokens. Thus, we introduce a scheduling window $\hat{N}$, where PipeSD performs pipeline scheduling of draft tokens in each speculative round with a granularity of $\hat{N}$ tokens. We dynamically adjust $\hat{N}$ based on the moving average length of the most recent 100 draft sequences ($\hat{N}$ is initialized to 20 in our experiments). Simultaneously, we establish two rules: (1) When a cloud NAV is triggered, the current pipeline scheduling period is interrupted and all unsent draft tokens are immediately transmitted in a single batch; (2) While awaiting cloud NAV results, the edge continues generating draft tokens and transmits them in batches with a period of $\hat{N}$ to further overlap computing and communication (see Appendix B for details).

Algorithm 1 DP for Optimal Token Batching

Input: $\hat{N}, \alpha, \beta, \gamma$
for $j = 1$ **to** $\hat{N}$ **do**
  $\mathrm{dp}[j] \leftarrow +\infty,\ \mathrm{prev}[j] \leftarrow \varnothing$
$\mathrm{dp}[0] \leftarrow 0$
for $j = 1$ **to** $\hat{N}$ **do**
  for $i = 0$ **to** $j - 1$ **do**
    $t_c \leftarrow \alpha + \beta \cdot (j - i)$ *// Eq. (2)*
    $temp \leftarrow \max\{\mathrm{dp}[i],\ \gamma \cdot j\} + t_c$ *// Eqs. (3)-(5)*
    if $temp < \mathrm{dp}[j]$ **then**
      $\mathrm{dp}[j] \leftarrow temp,\ \mathrm{prev}[j] \leftarrow i;$
// Backtrack
$\mathbb{B} \leftarrow (),\ p \leftarrow \hat{N}$
while $p > 0$ **do**
  $q \leftarrow \mathrm{prev}[p],\ \mathbb{B} \leftarrow (q + 1, \mathbb{B}),\ p \leftarrow q$
return $\mathbb{B}$

4. Algorithm and System Implementation

4.1. Algorithm Design for Optimal Token Batching

We adopt a DP approach to obtain the optimal token batching strategy under the proposed pipeline model. Algorithm 1 summarizes the DP procedure.

The algorithm takes as input the scheduling window $\hat{N}$ and the communication and computation parameters (α, β, γ), and returns the optimal batching strategy $\mathbb{B}$. First, it initializes a DP table $\mathrm{dp}[\cdot]$, where $\mathrm{dp}[j]$ represents the minimal autoregressive generation and communication time for the first j tokens ($j \in [1, \hat{N}]$) and $\mathrm{dp}[0] = 0$ is the base case for the empty prefix. Then, it iteratively fills the DP table by enumerating possible batch boundaries. Finally, $\mathbb{B}$ is recovered by backtracking through the DP table.

The time complexity of the DP algorithm is $O(\hat{N}^2)$, as it enumerates all pairs of batch boundaries (i, j) with $i < j$. In practice, $\hat{N}$ is typically small, making the DP overhead negligible. Moreover, Algorithm 1 is re-executed to update the token batching strategy only when the scheduling window $\hat{N}$ changes or when the parameters (α, β, γ) change substantially (see Appendix D.2 for details). Our experiments in Sec. 5.2.5 show that the DP overhead is less than 0.013% of the total time of 1000 speculative rounds.

We also formally state the optimality of the DP-based batching algorithm (The proof is provided in Appendix E).

Theorem 4.1. *Under the pipeline model defined in Sec. 3.2, Algorithm 1 returns an optimal token batching strategy* $\mathbb{B}$.

4.2. System Implementation

Figure 4 illustrates the system architecture of PipeSD, which consists of an edge device and a cloud server. The edge device performs draft token generation, transmission control, and dynamic environment-aware adaptation, while the cloud only starts a FastAPI server and executes NAV using the

已计算但尚未验证，并且 R_1 表示累计序列置信度阈值。当草稿模型自回归地生成 n- 第 D_n 个草稿 token 时，PipeSD 通过将 $P(D_n)$ 与当前 C_1 相乘来更新暂定累计置信度 C_1^*。如果 $C_1^* \leq R_1$，则会触发云 NAV，并且 C_1 被重置为 1。此外，我们使用 R_2 表示单个 token 的置信度阈值。如果 $P(D_n) \leq R_2$，也会触发云 NAV。因此，所提出的双阈值 NAV 触发机制共同考虑了整个草稿序列和单个 token 的置信度，有效地提高了 NAV 的灵活性。

值得注意的是，阈值对 (R_1, R_2) 与推理任务的难度密切相关，并且对协作推理的速度有显著影响。不幸的是，将每个被接受的生成 token 的平均生成时间 (TPT) 显式建模为 (R_1, R_2) 的函数并非易事。为了确保 PipeSD 的自适应性，我们设计了一个轻量级的 BO 自动调优器 来自动调整 (R_1, R_2)。

BO 自动调优器的目标是以尽可能少的样本，为未知目标函数寻找较优参数。在本文场景中，目标是最小化平均 TPT。自动调优器对不同三元组 (R_1, R_2, TPT) 进行采样，并持续建议下一个阈值对 (R_1, R_2) 以获取新的目标值。每个阈值对 (R_1, R_2) 对应的平均 TPT，通过多个已接受 token 的生成时间求平均获得。换言之，BO 自动调优器仅需少量样本便可高效预测接近最优的阈值。在实际推理中，只需 16 个样本即可返回接近最优的阈值对。性能评测和参数设置详见第 5.2.3 节与附录 C 。

需要说明的是，在 PipeSD 的双阈值触发机制下，每轮推测的草稿长度 N 会动态变化，并由生成 token 的置信度隐式决定。因此，我们引入调度窗口 $\hat{N}$，使 PipeSD 在每轮推测中以 $\hat{N}$ 个 token 为粒度进行流水线调度。我们根据最近 100 条草稿序列长度的移动平均值，动态调整 $\hat{N}$ （实验中 $\hat{N}$ 初始值为 20 ）。同时设定两项规则：(1) 触发云端 NAV 时，中断当前流水线调度周期，将所有尚未发送的草稿 token 立即作为单个批次传输；(2) 等待云端 NAV 结果期间，边缘继续生成草稿 token，并以 $\hat{N}$ 为周期分批传输，进一步重叠计算与通信，详见附录 B 。

Algorithm 1 DP for Optimal Token Batching

Input: $\hat{N}, \alpha, \beta, \gamma$
for $j = 1$ **to** $\hat{N}$ **do**
  $\mathrm{dp}[j] \leftarrow +\infty,\ \mathrm{prev}[j] \leftarrow \varnothing$
$\mathrm{dp}[0] \leftarrow 0$
for $j = 1$ **to** $\hat{N}$ **do**
  for $i = 0$ **to** $j - 1$ **do**
    $t_c \leftarrow \alpha + \beta \cdot (j - i)$ *// Eq. (2)*
    $temp \leftarrow \max\{\mathrm{dp}[i],\ \gamma \cdot j\} + t_c$ *// Eqs. (3)-(5)*
    if $temp < \mathrm{dp}[j]$ **then**
      $\mathrm{dp}[j] \leftarrow temp,\ \mathrm{prev}[j] \leftarrow i;$
// Backtrack
$\mathbb{B} \leftarrow (),\ p \leftarrow \hat{N}$
while $p > 0$ **do**
  $q \leftarrow \mathrm{prev}[p],\ \mathbb{B} \leftarrow (q + 1, \mathbb{B}),\ p \leftarrow q$
return $\mathbb{B}$

4. 算法与系统实现

4.1. 最优 token 批处理的算法设计

我们采用动态规划（ DP ）方法来获得在所提出的流水线模型下的最优 token 批处理策略。算法 1 总结了动态规划过程。

该算法以调度窗口 $\hat{N}$ 以及通信和计算参数（ α,β, γ ）作为输入，并返回最优的批处理策略 $\mathbb{B}$。首先，它初始化一个 DP 表 $\mathrm{dp}[\cdot]$，其中 $\mathrm{dp}[j]$ 表示前 j 个 tokens ($j \in [1, \hat{N}]$) 的最小自回归生成和通信时间，而 $\mathrm{dp}[0] = 0$ 是空前缀的基本情况。然后，它通过枚举可能的批次边界迭代地填充 DP 表。最后，$\mathbb{B}$ 是通过回溯 DP 表得到的。

DP 算法的时间复杂度为 $O(\hat{N}^2)$，因为它枚举所有满足 $i < j$ 的批次边界对 (i, j) 。实际应用中，$\hat{N}$ 通常较小，DP 开销可忽略。此外，仅在调度窗口 $\hat{N}$ 变化，或参数 (α, β, γ) 显著变化时，才重新执行算法 1 来更新 token 批次策略（详见附录 D.2 ）。第 5.2.5 节的实验表明，DP 开销占 1000 轮推测总时间的比例低于 0.013%。

我们还正式说明基于动态规划的批处理算法的最优性（证明见附录 E ）。

定理 4.1。在第 3.2 节定义的流水线模型下，算法 *1* 返回一个最优的 token 批处理策略 $\mathbb{B}$。

4.2. 系统实现

图 4 展示了 PipeSD 的系统架构，该架构由边缘设备和云服务器组成。边缘设备执行初稿 token 生成、传输控制和动态环境感知适应，而云端仅启动 FastAPI 服务器并使用 执行 NAV。

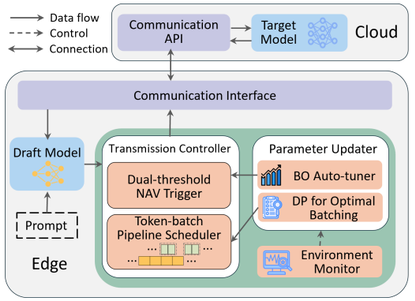


Figure 4. Overview of PipeSD architecture. The green part is the core of PipeSD.

target model, which makes the system easy to scale and compatible with existing cloud-edge collaborative frameworks. Moreover, although the current design only considers a single client, PipeSD can be easily extended to support multiple clients with minor modifications (see Appendix I for details).

The modules on the edge device include: (1) **Draft Model**, which generates draft tokens autoregressively, implemented with llama-cpp-python to enable efficient GGUF-based CPU inference (Hugging Face, 2023). Moreover, token-tree based drafting can also be enabled in PipeSD (see Appendix J for details); (2) **Transmission Controller**, which orchestrates pipeline scheduling and NAV triggering by *Token-batch Pipeline Scheduler* and *Dual-threshold NAV Trigger*, respectively; (3) **Communication Interface**, which uploads draft tokens and NAV requests to the cloud server, and receives NAV results; (4) **Environment Monitor**, which continuously monitors the average TPT and the communication and computation parameters (α, β, γ), and triggers targeted updates when significant changes are detected(see Appendix D for details); and (5) **Parameter Updater**, which re-runs the BO autotuner to update (R_1, R_2) upon significant TPT changes, and re-executes the DP scheduler (Algorithm 1) to update $\mathbb{B}$ upon significant changes in (α, β, γ) (see Appendix D for details).

The modules on the cloud include: (1) **Communication API**, which starts a FastAPI server and exposes endpoints for receiving draft tokens and NAV signals from the edge device, and returning NAV results; and (2) **Target Model**, which performs NAV on the received draft tokens.

5. Evaluation

5.1. Experimental Setups

Testbed. Our cloud-edge testbed is deployed in a real-world metropolitan network environment. The **edge device** is a Lenovo ThinkBook 16+ equipped with an Intel® Core™ Ultra 9 185H CPU (16 cores, up to 5.1 GHz) and 32 GB of system memory, running Windows 11 (24H2). The **cloud server** is hosted on Tianyi Cloud and equipped with an NVIDIA A800 GPU (40 GB VRAM), an Intel Xeon CPU, and 120 GB of system memory, running Ubuntu 22.04 LTS. The uplink and downlink bandwidths are 20 Mbps and 200 Mbps, respectively, which meet the standard 5G communication bandwidth (Wu et al., 2024). We construct four experimental scenarios. Scenarios 1–3 use the above static network setting but differ in the edge device compute capability. Specifically, Scenario 1 uses the above-mentioned Lenovo ThinkBook as the edge device, while Scenarios 2 and 3 emulate a mobile phone and an IoT device, respectively. We set the computing power of the simulated mobile phone and IoT device to 2.5 GHz and 1.2 GHz, respectively, which are common device frequencies (Liu et al., 2024; Fitzgibbon & Ottaviani, 2024). We emulate these devices by adding corresponding latency on the same testbed (see Appendix G.2 for emulation details). Scenario 4 uses the same edge device setting as Scenario 1 but applies a dynamic-bandwidth setting. In this scenario, the uplink and downlink bandwidths vary within $[10, 80]$ Mbps and $[150, 280]$ Mbps, respectively, with a change interval of 20 seconds (Al-Falahy & Alani, 2017) (see Appendix G.1 for bandwidth control details).

Models and Datasets. We evaluate PipeSD on programming and mathematical reasoning tasks. For programming, we use the HumanEval dataset (Chen et al., 2021), with DeepSeek-Coder-1.3B as the draft model and DeepSeek-Coder-6.7B as the target model (Guo et al., 2024). For mathematical reasoning, we use GSM8K (Cobbe et al., 2021), where TinyLlama-1.1B-Chat-v1.0 (Zhang et al., 2024) and Llama-2-7B (Touvron et al., 2023b) serve as the draft and target models, respectively.

Baselines. Some pioneering works are orthogonal to PipeSD: HAT studies cloud-edge collaborative speculative decoding under relaxed accuracy constraints, while SpecEdge focuses on multi-edge collaboration. We benchmark PipeSD against the following representative frameworks: (1) Vanilla Cloud-Edge Collaborative Speculative Decoding (Vanilla) (Kim et al., 2023), where the edge device autoregressively generates and uploads a fixed number of draft tokens, we set $N = 6$ for programming tasks and $N = 4$ for mathematical reasoning tasks, which yield the best performance across all scenarios. (2) HSL (Hao et al., 2024), which triggers cloud NAV when the confidence of a draft token falls below a predefined threshold. We set the thresholds to 0.99 for programming tasks and 0.7 for mathematical reasoning tasks for HSL's best performance across all scenarios. (3) EdgeLLM (Xu et al., 2025), which is adapted for the cloud-edge collaboration scenario while retaining its two key mechanisms: (i) continuing draft gener-

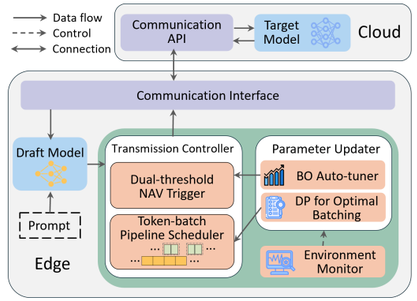


图 4．PipeSD 架构概览。绿色部分是 PipeSD 的核心。

目标模型，使系统易于扩展且兼容现有的云 - 边缘协作框架。此外，尽管当前设计仅考虑单个客户端，PipeSD 可以通过小幅修改轻松扩展以支持多个客户端（详情见附录 I ）。

边缘设备上的模块包括：(1) 草稿模型，用于自回归生成草稿 token，使用 llama-cpp-python 实现，以支持高效的基于 GGUF 的 CPU 推理（ Hugging Face, 2023 ）。此外，在 PipeSD 中也可以启用基于 tokentree 的草稿生成（详情参见附录 J ）；(2) 传输控制器，由 Token-batch Pipeline Scheduler 和 双阈值 NAV Trigger 分别协调流水线调度和 NAV 触发；(3) 通信接口，用于将草稿 token 和 NAV 请求上传至云服务器，并接收 NAV 结果；(4) 环境监控器，持续监控平均 TPT 以及通信和计算参数 (α, β, γ)，在检测到显著变化时触发针对性更新（详情参见附录 D ）；以及 (5) 参数更新器，当 TPT 发生显著变化时，会重新运行 BO 自动调优器以更新 (R_1, R_2)，并在 (α, β, γ)发生显著变化时重新执行 DP 调度器（算法 1 ）以更新 $\mathbb{B}$ （详见附录 D ）。

云上的模块包括：(1) **通信 API** ，它启动一个 FastAPI 服务器并开放端点，用于接收来自边缘设备的草稿 token 和 NAV 信号，并返回 NAV 结果；以及 (2) **目标模型** ，它对接收到的草稿 token 执行 NAV。

5. 评估

5.1. 实验设置

测试平台。我们的云边测试平台部署在现实世界的都市网络环境中。**边缘设备**是一个联想 ThinkBook 16+ 配备英特尔 ® 核心 ™Ultra 9 185H CPU （ 16 核，最高 5.1 GHz ）和 32 GB 系统内存，运行 Windows 11 （ 24H2 ）。 **云服务器** 托管在天翼云上，配备 NVIDIA A800 GPU （ 40 GB 显存）、 Intel Xeon CPU 和 120 GB 系统内存，运行 Ubuntu 22.04 LTS。上行和下行带宽分别为 20 Mbps 和 200 Mbps，符合标准 5G 通信带宽 Wu et al., 2024 我们构建了四个实验场景。场景 1–3 使用上述静态网络设置，但边缘设备的计算能力不同。具体来说，场景 1 使用上述提到的联想 ThinkBook 作为边缘设备，而场景 2 和 3 分别模拟一部手机和一种物联网设备。我们将模拟手机和物联网设备的计算能力分别设置为 2.5 GHz 和 1.2 GHz，这是常见的设备频率。Liuet al.,2024; Fitzgibbon & Ottaviani, 2024)。我们通过在相同的测试平台上添加相应的时延来模拟这些设备（见附录 G.2 用于仿真详情）。场景 4 使用与场景 1 相同的边缘设备设置，但应用了动态带宽设置。在此场景中，上行和下行带宽会在 $[10, 80]$ Mbps 和 $[150, 280]$ Mbps，分别，以 20 秒的变化间隔（阿尔－法拉希与阿拉尼 , 2017）（见附录 G.1 有关带宽控制的详细信息)。

模型和数据集。 我们在编程和数学推理任务上评估 PipeSD。对于编程，我们使用 HumanEval 数据集（ Chen et al., 2021), 使用 DeepSeek-Coder-1.3B 作为草稿模型, DeepSeek-Coder-6.7B 作为目标模型 (Guo et al., 2024). 对于数学推理，我们使用 GSM8K （ Cobbe etal., 2021), 其中 TinyLlama-1.1B-Chat-v1.0 (Zhang et al., 2024) 和 Llama-2-7B (Touvron et al., 2023b) 分别作为草稿和目标模型。

基线。 一些开创性的工作与 PipeSD 无关：HAT 研究在放宽精度约束下的边云协同推测解码，而 SpecEdge 专注于多边缘协作。我们将 PipeSD 与以下代表性框架进行基准测试：(1) 原生边云协同推测解码 (Vanilla) (Kim etal., 2023 ）， 其中边缘设备自回归地生成并上传固定数量的草稿 token，我们设置 $N = 6$ 用于编程任务和 $N = 4$ 对于数学推理任务，在所有场景中产生最佳性能。(2) HSL （ Haoetal.,2024 ），当草稿 token 的置信度低于预定义阈值时，会触发云端 NAV。我们将阈值设置为 0.99 用于编程任务和 0.7 用于数学推理任务，以实现 HSL 在所有场景中的最佳性能。(3) EdgeLLM (Xuet al., 2025)，它被改编用于边云协同场景，同时保留其两个关键机制：（ i ）持续的草稿生成

Table 1. Comparison of average TPT (ms). **ID** denotes the scenario index. S_{t1}, S_{t2}, and S_{t3} denote the speedup of PipeSD over Vanilla, HSL, and EdgeLLM, respectively.

ID	Dataset	TPT (ms)				Speedup		
		Vanilla	HSL	EdgeLLM	PipeSD	S_{t1}	S_{t2}	S_{t3}
1	HumanEval	194	155	153	129	1.50×	1.20×	1.19×
	GSM8K	193	174	169	145	1.33×	1.20×	1.17×
2	HumanEval	225	184	166	134	1.68×	1.37×	1.24×
	GSM8K	318	223	197	168	1.89×	1.33×	1.17×
3	HumanEval	306	244	201	152	2.01×	1.61×	1.32×
	GSM8K	402	296	231	186	2.16×	1.59×	1.24×
4	HumanEval	160	132	127	108	1.48×	1.22×	1.18×
	GSM8K	234	165	161	139	1.68×	1.19×	1.16×

Table 2. Comparison of ECS (J) in Scenario 1. P_{e1}, P_{e2}, and P_{e3} denote the ECS reduction of PipeSD compared to Vanilla, HSL, and EdgeLLM, respectively.

Dataset	ECS (J)				ECS Reduction (%)		
	Vanilla	HSL	EdgeLLM	PipeSD	P_{e1}	P_{e2}	P_{e3}
HumanEval	68	71	75	56	17.6	21.1	25.3
GSM8K	98	102	100	84	14.3	17.6	16.0

ation while waiting for NAV, and (ii) triggering NAV when the cumulative sequence confidence falls below a dynamically adjusted threshold (see Appendix G.3 for details).

Metrics. We focus on two metrics: (i) the average generation time per accepted token (TPT), and (ii) the average energy consumption of the cloud server per 100 accepted tokens (ECS). All reported results are averaged over 1000 accepted tokens.

5.2. Experimental Results

5.2.1. Overall Performance Comparison

TPT Comparison: Table 1 reports the average TPT of all methods on both datasets across four scenarios. The results show that PipeSD achieves speedups of 1.33×–2.16×, 1.19×–1.61×, and 1.16×–1.32× compared to Vanilla, HSL, and EdgeLLM, respectively. For Scenarios 2–3 with limited computing power of the edge, PipeSD attains larger gains because more communication time can be hidden by token-batch pipelining. Under dynamic bandwidth (Scenario 4), PipeSD still achieves consistent improvements, demonstrating its robustness to bandwidth fluctuations (see Appendix D for more details). Overall, performance improvement mainly comes from two mechanisms of PipeSD: token-batch pipeline scheduling overlaps computing and communication to hide communication latency, while dual-threshold NAV triggering enables timely verification, avoiding both premature and delayed triggering. **ECS Comparison:** We compute ECS by time-integrating the cloud

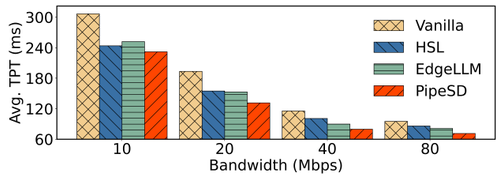


Figure 5. Average TPT (ms) with different bandwidth levels on HumanEval in Scenario 1.

Table 3. Performance evaluation of BO autotuner in Scenario 1.

Dataset	TPT (ms)		
	BO Autotuner	Grid Search	Random Search
HumanEval	129	139	148
GSM8K	145	155	162

GPU power trace sampled with NVIDIA SMI (NVIDIA Corporation, 2025) at 5 ms intervals (Gao et al., 2025). Table 2 shows the ECS of all methods on both datasets in Scenario 1. Compared to Vanilla, HSL, and EdgeLLM, PipeSD achieves ECS reductions of 17.6%, 21.1%, and 25.3% on HumanEval, and 14.3%, 17.6%, and 16.0% on GSM8K, respectively. This improvement is mainly attributed to the more accurate and efficient verification triggering brought by the dual-threshold NAV triggering mechanism, which effectively reduces unnecessary verification requests. While ECS captures cloud-side energy, it is also important to assess whether PipeSD incurs additional energy overhead on the edge. However, accurately measuring edge-side energy is challenging, because the edge runs inference on a general-purpose CPU and the measured power can be affected by background OS activities and other software processes. We therefore provide a theoretical analysis of the edge-side energy consumption in Appendix H, which indicates that PipeSD introduces only negligible additional edge energy overhead from BO autotuning, DP scheduling, and parameter measurement.

5.2.2. Impact of Network Bandwidth

Figure 5 shows the performance of PipeSD under different uplink bandwidths on HumanEval in Scenario 1. At 10, 20, 40, and 80 Mbps, PipeSD accelerates inference by 1.32×, 1.47×, 1.45×, and 1.34× over Vanilla; 1.05×, 1.18×, 1.26×, and 1.21× over HSL; and 1.09×, 1.16×, 1.13×, and 1.14× over EdgeLLM, respectively. We observe that the average TPT stabilizes as the bandwidth reaches 80 Mbps. This is because, at higher bandwidth levels, communication is no longer the performance bottleneck.

表 1. 平均 TPT (毫秒) 的比较。**ID** 表示场景索引。 S_{t1}、 S_{t2} 和 S_{t3} 分别表示 PipeSD 相对于 Vanilla 、 HSL 和 EdgeLLM 的加速比。

ID	Dataset	TPT (ms)				Speedup		
		Vanilla	HSL	EdgeLLM	PipeSD	S_{t1}	S_{t2}	S_{t3}
1	HumanEval	194	155	153	129	1.50×	1.20×	1.19×
	GSM8K	193	174	169	145	1.33×	1.20×	1.17×
2	HumanEval	225	184	166	134	1.68×	1.37×	1.24×
	GSM8K	318	223	197	168	1.89×	1.33×	1.17×
3	HumanEval	306	244	201	152	2.01×	1.61×	1.32×
	GSM8K	402	296	231	186	2.16×	1.59×	1.24×
4	HumanEval	160	132	127	108	1.48×	1.22×	1.18×
	GSM8K	234	165	161	139	1.68×	1.19×	1.16×

表 2 场景 1 中 ECS(J) 的比较。 P_{e1}, P_{e2}, 和 P_{e3} 分别表示 PipeSD 相对于 Vanilla 、 HSL 和 EdgeLLM 的 ECS 减少量。

Dataset	ECS (J)				ECS Reduction (%)		
	Vanilla	HSL	EdgeLLM	PipeSD	P_{e1}	P_{e2}	P_{e3}
HumanEval	68	71	75	56	17.6	21.1	25.3
GSM8K	98	102	100	84	14.3	17.6	16.0

在等待 NAV 的同时进行 ation，以及 (ii) 当累计序列置信度低于动态调整的阈值时触发 NAV （详细信息见附录 G.3 ）。

指标。我们关注两个指标：（ i ）每个被接受的 token 的平均生成时间（ TPT ），以及（ ii ）云服务器每 100 个被接受 token 的平均能耗（ ECS ）。所有报告的结果均为 1000 个被接受 token 的平均值。

5.2. 实验结果

5.2.1. 总体性能比较

TPT 比较：表格 1 报告了所有方法在两个数据集上的四种场景下的平均 TPT。结果显示，PipeSD 相比 Vanilla 、 HSL 和 EdgeLLM 分别实现了 1.33×–2.16×、1.19×–1.61×和 1.16×–1.32× 的加速。在边缘计算能力有限的场景 2–3 中，PipeSD 实现了更大的提升，因为通过 token 批处理流水线可以隐藏更多的通信时间。在动态带宽（场景 4 ）下，PipeSD 仍然实现了一致的改进，展示了其对带宽波动的鲁棒性（见附录 ）。 有关更多细节)。整体而言，性能提升主要来源于 PipeSD 的两种机制：token 批次流水线调度通过重叠计算与通信来隐藏通信时延，而双阈值 NAV 触发则实现及时验证，避免过早或时延触发。**ECS 比较：**我们通过对云的时间积分来计算 ECS

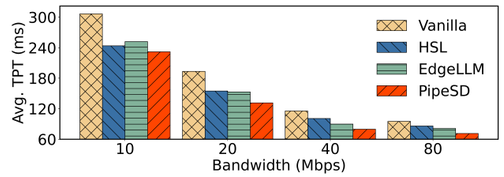


图 5. 在人类评估场景 1 中不同带宽水平下的平均 TPT （毫秒）。

表 3. 场景 1 中 BO 自动调优器的性能评估。

Dataset	TPT (ms)		
	BO Autotuner	Grid Search	Random Search
HumanEval	129	139	148
GSM8K	145	155	162

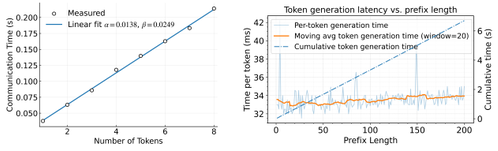
GPU 功耗轨迹使用 NVIDIA SMI 采样 (NVIDIACorporation, 2025) ， 采样间隔为 5 毫秒 (Gao 等人 , 2025)。表 2 显示了场景 1 中两个数据集上所有方法的 ECS。与 Vanilla 、 HSL 和 EdgeLLM 相比，PipeSD 在 HumanEval 上分别实现了 ECS 降低 17.6 % 、 21.1% 和 25.3% ， 而在 14.3 上分别为 % 、 17.6 % 和 16.0% 。 GSM8K，分别。这种改进主要归因于双阈值 NAV 触发机制带来的更准确高效的验证触发，有效减少了不必要的验证请求。虽然 ECS 捕获了云端的能量，但评估 PipeSD 是否在边缘端产生额外的能量开销也很重要。然而，准确测量边缘侧的能量非常具有挑战性，因为边缘端在通用 CPU 上运行推理，而测量到的功耗可能会受到后台操作系统活动和其他软件进程的影响。因此，我们在附录 H 中提供了边缘侧能量消耗的理论分析，结果表明 PipeSD 在 BO 自动调优、 DP 调度和参数方面仅引入了微不足道的额外边缘能量开销。 measurement.

5.2.2. 网络带宽的影响

图 5 显示了 PipeSD 在场景 1 中不同上行带宽下对 HumanEval 的性能。在 10 、 20 、 40 和 80 Mbps 时，PipeSD 对比 Vanilla 提速分别为 1.32×、 1.47×、 1.45× 和 1.34× ； 对比 HSL 提速分别为 1.05×、1.18×、 1.26× 和 1.21× ； 对比 EdgeLLM 提速分别为 1.09×、 1.16×、1.13×和 1.14×。我们观察到，当带宽达到 80 Mbps 时，平均 TPT 稳定下来。 因为在更高的带宽水平下，通信不再是性能瓶颈。

Table 4. Comparison of TPT (ms) when using PipeSD with BO autotuner or different fixed (R_1, R_2) on HumanEval in Scenario 1.

	TPT (ms)								
BO	(0.3, 0.3)	(0.3, 0.6)	(0.3, 0.9)	(0.6, 0.3)	(0.6, 0.6)	(0.6, 0.9)	(0.9, 0.3)	(0.9, 0.6)	(0.9, 0.9)
129	197	174	139	189	174	139	149	156	138



(a) Communication time vs. number of tokens in a batch. *(b)* Per token generation time γ with respect to the prefix length.

Figure 6. Communication and computation latency characteristics used in PipeSD.

Table 5. Percentage of the overhead of BO autotuner, DP scheduler and parameter measurement to the total time of first 1000 speculative rounds in Scenario 1.

Dataset	HumanEval	GSM8K
Overhead of BO autotuner	1.1%	0.9%
Overhead of DP scheduler	0.01%	0.013%
Overhead of parameter measurement	0.3%	0.4%

Table 6. Ablation studies on HumanEval in Scenario 1.

Method	Pipeline	NAV trigger	TPT (ms)	Speedup
Vanilla	✗	Fixed-length	194	1.00×
PipeSD w/o Pipeline	✗	Dual-threshold	147	1.32×
PipeSD + Fixed-length	✓	Fixed-length	164	1.18×
PipeSD + Token-level	✓	Token-level	137	1.42×
PipeSD + Sequence-level	✓	Sequence-level	139	1.40×
PipeSD (Full)	✓	Dual-threshold	129	1.50×

5.2.3. Performance Evaluation of BO Autotuner

We evaluate the effectiveness of BO autotuner by comparing it with grid search and random search on tuning (R_1, R_2). As shown in Table 3, BO autotuner consistently achieves the lowest average TPT on both HumanEval and GSM8K. Compared with grid search and random search, BO is able to efficiently converge to near-optimal configurations with minimal overhead, making it more suitable for online parameter optimization (see Appendix C.2 for more analysis). In addition, we compare the average TPT when using PipeSD with BO autotuner or fixed (R_1, R_2) pairs on HumanEval in Scenario 1. As shown in Table 4, different (R_1, R_2) greatly affect the inference efficiency and BO autotuning is essential to maximize the performance of PipeSD.

5.2.4. Parameter Measurement

Measurement for α and β: We measure the communication latency for transmitting token batches of different sizes. As shown in Figure 6a, the communication time increases linearly with the number of tokens in a batch, where the intercept and slope of the fitted line correspond to α and β, respectively. **Measurement for γ:** Figure 6b shows that, within a 200-token sliding window, γ remains approximately constant across different prefix lengths.

5.2.5. Overhead Analysis

We measure the computational overhead of the BO autotuner, DP scheduler and parameter measurement by profiling their runtime during the first 1000 speculative rounds on both datasets in Scenario 1. Table 5 reports their overhead as a percentage of the total execution time. The results show that the overhead of the BO autotuner is negligible, accounting for no more than 1.1% on both datasets, which underscores its efficiency as a parameter-tuning mechanism. In addition, the overhead of parameter measurement contributes less than 0.4%, as it is executed only when significant environmental changes are detected. The overhead of the DP scheduler is negligible (below 0.013%).

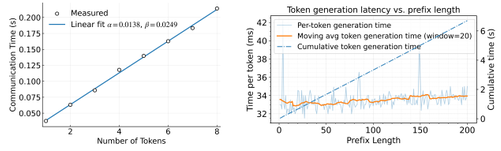
5.2.6. Ablation Studies

We conduct ablation studies on HumanEval in Scenario 1. The results are shown in Table 6, where (1) *PipeSD w/o Pipeline* disables the pipeline scheduling and instead transmits draft tokens after generating the entire draft sequence, (2) *PipeSD + Fixed-length* adopts a fixed-length NAV triggering strategy, (3) *PipeSD + Token-level* uses a single-token confidence-based strategy, and (4) *PipeSD + Sequence-level* adopts a cumulative sequence confidence-based strategy. It is seen that *PipeSD w/o Pipeline* is 1.12× slower than the full PipeSD, which demonstrates the effectiveness of token-batch pipeline scheduling mechanism. Meanwhile, compared to *PipeSD + Fixed-length*, *PipeSD + Token-level*, and *PipeSD + Sequence-level*, the full PipeSD achieves speedups of 1.25×, 1.05×, and 1.06×, respectively, which indicates that jointly considering both single-token and cumulative sequence confidence provides more effective NAV triggering.

To further verify that the benefit of the pipeline mechanism does not merely come from introducing pipelining, but also from the proposed DP-based token-batching policy, we compare the DP-based token-batching policy with stronger pipelined baselines, including a greedy policy that sends all accumulated tokens whenever the network becomes idle, as well as two heuristic policies, namely immediate-send and

表 4 使用 PipeSD 搭配 BO 自动调优器或不同 xed 时的 TPT （毫秒）比较 R_1, R_2) 在场景 1 中的 HumanEval 上。

	TPT (ms)								
BO	(0.3, 0.3)	(0.3, 0.6)	(0.3, 0.9)	(0.6, 0.3)	(0.6, 0.6)	(0.6, 0.9)	(0.9, 0.3)	(0.9, 0.6)	(0.9, 0.9)
129	197	174	139	189	174	139	149	156	138



(a) 通信时间与批次中的 token 数量。*(b)* 每个 token 生成时间 γ 关于前 x 长度。图 6。 PipeSD 中使用的通信和计算时延特性。

表 5。 在场景 1 中， BO 自动调优器、 DP 调度器和参数测量的开销占前 1000 轮推测总时间的百分比。

Dataset	HumanEval	GSM8K
Overhead of BO autotuner	1.1%	0.9%
Overhead of DP scheduler	0.01%	0.013%
Overhead of parameter measurement	0.3%	0.4%

表 6. 在人类评估场景 1 的消融研究。

Method	Pipeline	NAV trigger	TPT (ms)	Speedup
Vanilla	✗	Fixed-length	194	1.00×
PipeSD w/o Pipeline	✗	Dual-threshold	147	1.32×
PipeSD + Fixed-length	✓	Fixed-length	164	1.18×
PipeSD + Token-level	✓	Token-level	137	1.42×
PipeSD + Sequence-level	✓	Sequence-level	139	1.40×
PipeSD (Full)	✓	Dual-threshold	129	1.50×

5.2.3. BO 自动调优器的性能评估

我们通过将 BO 自动调优器与网格搜索和随机搜索在调优上的表现进行比较来评估其有效性 (R_1, R_2). 如表所示 3, BO 自动调优器在 HumanEval 和 GSM8K 上始终实现了最低的平均 TPT。与网格搜索和随机搜索相比， BO 能够以最小的开销高效地收敛到接近最优的配置，使其更适合在线参数优化（见附录 C.2 以便进行更多分析）。此外，我们比较了在使用 PipeSD 搭配 BO 自动调优器或固定值时的平均 TPT (R_1, R_2) 在场景 1 中的人类评估对。如表所示 4， 不同 (R_1, R_2) 极大地影响推理效率， BO 自动调优对于最大化 PipeSD 的性能是必不可少的。

5.2.4. 参数测量

对 α 和 β 的测量： 我们测量了不同大小 token 批次传输的通信时延。如图 6a 所示，通信时间随批次中 token 数线性增加，其中拟合直线的截距和斜率分别对应 α 和 β。 **对 γ 的测量：** 图 6b 显示，在 200 个 token 的滑动窗口内， γ 在不同前缀长度下保持大致恒定。

5.2.5. 间接费用分析

我们通过在场景 1 中对两个数据集的前 1000 次投机轮的运行时间进行分析，来测量 BO 自动调优器、 DP 调度器和参数测量的计算开销。表 5 显示了它们作为总执行时间百分比的开销。结果表明， BO 自动调优器的开销可以忽略不计，

5.2.6. 消融研究

我们在场景 1 中对 HumanEval 进行了消融实验。结果如表所示 6， 其中 (1) PipeSD 无流水线 禁用流水线调度，而是在生成整个草稿序列后传输草稿 token, (2) *PipeSD* + 固定长度 采用固定长度的净值触发策略, (3)*PipeSD* + token 级 使用单一 token 的基于置信度的策略，以及 (4) *PipeSD* + 序列级 采用累积序列置信度策略。可以看到 PipeSD 无流水线 is 1.12× 比完整的 PipeSD 更慢，这证明了 token 批次流水线调度机制的有效性。与此同时，相比于 *PipeSD* + 固定长度 , *PipeSD* +token 级，和 *PipeSD* + 序列级，完整的 PipeSD 实现了加速 1.25×, 1.05×，和 1.06×，分别，这表明联合考虑单个 token 和累积序列的置信度可以更有效地触发 NAV。

在两个数据集中均不超过 1.1%，这凸显了其作为参数调优机制的高效性。此外，参数测量的开销不到 0.4 %，因为它仅在检测到显著环境变化时执行。DP 调度器的开销可以忽略不计（低于 0.013% ）。

为了进一步验证流水线机制的好处不仅仅来自于引入流水线，还来自于所提出的基于 DP 的 token 批处理策略，我们将基于 DP 的 token 批处理策略与更强的流水线基准进行比较，包括一种贪婪策略，该策略在网络空闲时发送所有累积的 token，以及两种启发式策略，即即时发送和

Table 7. Speculative-decoding statistics on HumanEval in Scenario 1.

Method	Verification Frequency	Mean Draft Length	Acceptance Rate
HSL	0.2558	3.18	0.9148
EdgeLLM	0.1912	4.74	0.8917
PipeSD	0.1733	4.96	0.9616

no-early-upload. The detailed setup and results are reported in Appendix F, where DP consistently achieves the best performance with negligible scheduling overhead.

To further understand the advantage of the dual-threshold NAV triggering mechanism, we additionally report several speculative-decoding statistics on HumanEval in Scenario 1, which provide a more fine-grained view of verification behavior beyond TPT alone. Table 7 provides a more detailed view of the NAV-triggering behavior of different methods. HSL is overly conservative with frequent verification and short draft sequences, limiting speculative gains. EdgeLLM is more balanced, but still achieves a lower acceptance rate than PipeSD. PipeSD achieves the best trade-off by maintaining a moderate draft length while attaining the highest acceptance rate. This suggests that the dual-threshold NAV triggering mechanism enables more accurate verification decisions, thereby reducing end-to-end inference latency.

6. Conclusion

In this paper, we propose PipeSD, an efficient cloud-edge collaborative inference framework with speculative decoding. First, PipeSD introduces a token-batch pipeline scheduling mechanism that overlaps draft token generation and communication, and leverages a DP algorithm to obtain optimal token batching strategies, thereby improving overall resource utilization. Second, PipeSD employs a dual-threshold NAV triggering mechanism to enhance verification flexibility, and incorporates a lightweight BO autotuner to automatically adjust thresholds. We implement PipeSD using llama-cpp-python, PyTorch, and FastAPI, and evaluate it on real-world cloud-edge testbeds. Extensive experiments with two draft-target model pairs across four scenarios demonstrate the superiority of PipeSD over state-of-the-art baselines, including HSL and EdgeLLM, achieving 1.16×–2.16× speedup and reducing energy consumption by 14.3%–25.3%. Future work will validate PipeSD's robustness in more diverse real-world settings, including varying network conditions, hardware platforms, and task types.

Acknowledgements

This work was supported in part by the Zhongguancun Academy, (Grant No.s XTS0038), and in part by Information Technology Center and State Key Lab of CAD&CG, ZheJiang University.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

Al-Falahy, N. and Alani, O. Y. Technologies for 5g networks: Challenges and opportunities. *IT Professional*, 19 (1):12–20, 2017. doi: 10.1109/MITP.2017.9.

Alipourfard, O., Liu, H. H., Chen, J., Venkataraman, S., Yu, M., and Zhang, M. Cherrypick: adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation*, NSDI'17, pp. 469–482, USA, 2017. USENIX Association. ISBN 9781931971379.

Betlen, Andrei. llama-cpp-python: Python bindings for llama.cpp. https://github.com/abetlen/llama-cpp-python, 2023.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., et al. Language models are few-shot learners, 2020. URL https://arxiv.org/abs/2005.14165.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. URL https://arxiv.org/abs/2401.10774.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023. URL https://arxiv.org/abs/2302.01318.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., et al. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL https://arxiv.org/abs/2205.14135.

表 7. 场景 1 下 HumanEval 的推测解码统计。

Method	Verification Frequency	Mean Draft Length	Acceptance Rate
HSL	0.2558	3.18	0.9148
EdgeLLM	0.1912	4.74	0.8917
PipeSD	0.1733	4.96	0.9616

no-early-upload。详细的设置和结果在附录 F 中报告，其中 DP 一直以可忽略的调度开销实现最佳性能。

为了进一步理解双阈值 NAV 触发机制的优势，我们另外报告了 Scenario 1 中 HumanEval 的一些推测解码统计数据，这提供了比单纯 TPT 更细粒度的验证行为视角。表 7 提供了不同方法 NAV 触发行为的更详细视图。HSL 过于保守，经常进行验证且草稿序列较短，从而限制了推测性收益。EdgeLLM 更加平衡，但其接受率仍低于 PipeSD。PipeSD 通过维持适中的草稿长度同时实现最高接受率，达到了最佳权衡。这表明双阈值 NAV 触发机制能够做出更准确的验证决策，从而减少端到端推理时延。

6. 结论

在本文中，我们提出了 PipeSD，一种高效的边云协同推理框架，具有推测解码功能。首先，PipeSD 引入了一种 token 批次流水线调度机制，可以实现草稿 token 生成与通信的重叠，并利用 DP 算法获取最优的 token 批次策略，从而提高整体资源利用率。其次，PipeSD 采用双阈值 NAV 触发机制以增强验证灵活性，并结合轻量级 BO 自动调优器自动调整阈值。我们使用 llama-cpp-python 、 PyTorch 和 FastAPI 实现了 PipeSD，并在真实的云边测试环境中对其进行了评估。在两个草稿 - 目标模型组合的四种场景下进行的大量实验表明，PipeSD 优于包括 HSL 和 EdgeLLM 在内的最先进基线，实现了 1.16×–2.16× 的加速并降低了能耗。 消耗降低了 14.3%–25.3%。未来的工作将验证 PipeSD 在更多样化的真实环境中的稳健性，包括不同的网络条件、硬件平台和任务类型。

致谢

这项工作部分得到了中关村学院的支持（资助编号 XTS0038 ），并部分得到了 Informa 的支持

浙江大学信息技术中心与 CAD&CG 国家重点实验室。

影响声明

本文介绍的工作旨在推动机器学习领域的发展。我们的工作可能带来许多潜在的社会影响，但我们认为这里无需特别强调其中的任何一种。

参考文献

Al-Falahy, N. and Alani, O. Y. Technologies for 5g networks: Challenges and opportunities. *IT Professional*, 19 (1):12–20, 2017. doi: 10.1109/MITP.2017.9.

Alipourfard, O., Liu, H. H., Chen, J., Venkataraman, S., Yu, M., and Zhang, M. Cherrypick: adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation*, NSDI'17, pp. 469–482, USA, 2017. USENIX Association. ISBN 9781931971379.

Betlen, Andrei. llama-cpp-python: Python bindings for llama.cpp. https://github.com/abetlen/llama-cpp-python, 2023.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., et al. Language models are few-shot learners, 2020. URL https://arxiv.org/abs/2005.14165.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024. URL https://arxiv.org/abs/2401.10774.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023. URL https://arxiv.org/abs/2302.01318.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., et al. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL https://arxiv.org/abs/2205.14135.

FastAPI Developers. Fastapi. https://fastapi.tiangolo.com/, 2018.

Fitzgibbon, G. and Ottaviani, C. Constrained device performance benchmarking with the implementation of post-quantum cryptography. *Cryptography*, 8:21, 05 2024. doi: 10.3390/cryptography8020021.

Gao, Y., Hu, B., Mashhadi, M. B., Jin, A.-L., Xiao, P., and Wu, C. Us-byte: An efficient communication framework for scheduling unequal-sized tensor blocks in distributed deep learning. *IEEE Transactions on Parallel and Distributed Systems*, 35(1):123–139, 2024. doi: 10.1109/TPDS.2023.3331372.

Gao, Y., Hu, B., Mashhadi, M. B., Jin, A.-L., Zhang, Y., Xiao, P., Tafazolli, R., and Debbah, M. Flowmoe: A scalable pipeline scheduling framework for distributed mixture-of-experts training, 2025. URL https://arxiv.org/abs/2510.00207.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., et al. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024. URL https://arxiv.org/abs/2401.14196.

Hähnel, M., Döbel, B., Völp, M., and Härtig, H. Measuring energy consumption for short code paths using rapl. *SIGMETRICS Perform. Eval. Rev.*, 40(3): 13–17, January 2012. ISSN 0163-5999. doi: 10.1145/2425248.2425252. URL https://doi.org/10.1145/2425248.2425252.

Hao, Z., Jiang, H., Jiang, S., Ren, J., and Cao, T. Hybrid slm and llm for edge-cloud collaborative inference. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*, EdgeFM '24, pp. 36–41, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706639. doi: 10.1145/3662006.3662067. URL https://doi.org/10.1145/3662006.3662067.

Hockney, R. W. The communication challenge for mpp: Intel paragon and meiko cs-2. *Parallel Comput.*, 20:389–398, 1994. URL https://api.semanticscholar.org/CorpusID:22986998.

Holmes, C., Tanaka, M., Wyatt, M., Awan, A. A., Rasley, J., Rajbhandari, S., Aminabadi, R. Y., et al. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference, 2024. URL https://arxiv.org/abs/2401.08671.

Huang, K., Guo, X., and Wang, M. Specdec++: Boosting speculative decoding via adaptive candidate lengths, 2025. URL https://arxiv.org/abs/2405.19715.

Hubert, B. tc(8) — linux manual page. https://man7.org/linux/man-pages/man8/tc.8.html, 2001. Accessed: 2026-01-25.

Hugging Face. Gguf: A binary model file format for efficient loading and inference. https://huggingface.co/docs/hub/gguf, 2023.

Kim, S., Mangalam, K., Moon, S., Malik, J., Mahoney, M. W., Gholami, A., and Keutzer, K. Speculative decoding with big little decoder, 2023. URL https://arxiv.org/abs/2302.07863.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pp. 19274–19286, 2023.

Li, S., Wang, H., Xu, W., Zhang, R., Guo, S., Yuan, J., Zhong, X., Zhang, T., and Li, R. Collaborative inference and learning between edge slms and cloud llms: A survey of algorithms, execution, and open challenges, 2025a. URL https://arxiv.org/abs/2507.16731.

Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: Speculative sampling requires rethinking feature uncertainty, 2025b. URL https://arxiv.org/abs/2401.15077.

Liu, J., Zeng, Q., Xu, H., Xu, Y., Wang, Z., and Huang, H. Adaptive block-wise regularization and knowledge distillation for enhancing federated learning. *IEEE/ACM Transactions on Networking*, 32(1):791–805, 2024. doi: 10.1109/TNET.2023.3301972.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, pp. 932–949. ACM, April 2024. doi: 10.1145/3620666.3651335. URL http://dx.doi.org/10.1145/3620666.3651335.

NVIDIA. TensorRT-LLM speculative decoding. https://nvidia.github.io/TensorRT-LLM/advanced/speculative-decoding.html, 2025. Accessed: 2026-01-11.

FastAPI Developers. Fastapi. https://fastapi.tiangolo.com/, 2018.

Fitzgibbon, G. and Ottaviani, C. Constrained device performance benchmarking with the implementation of post-quantum cryptography. *Cryptography*, 8:21, 05 2024. doi: 10.3390/cryptography8020021.

Gao, Y., Hu, B., Mashhadi, M. B., Jin, A.-L., Xiao, P., and Wu, C. Us-byte: An efficient communication framework for scheduling unequal-sized tensor blocks in distributed deep learning. *IEEE Transactions on Parallel and Distributed Systems*, 35(1):123–139, 2024. doi: 10.1109/TPDS.2023.3331372.

Gao, Y., Hu, B., Mashhadi, M. B., Jin, A.-L., Zhang, Y., Xiao, P., Tafazolli, R., and Debbah, M. Flowmoe: A scalable pipeline scheduling framework for distributed mixture-of-experts training, 2025. URL https://arxiv.org/abs/2510.00207.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., et al. The llama 3 herd of models, 2024. URL https://arxiv.org/abs/2407.21783.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., et al. Deepseek-coder: When the large language model meets programming – the rise of code intelligence, 2024. URL https://arxiv.org/abs/2401.14196.

Hähnel, M., Döbel, B., Völp, M., and Härtig, H. Measuring energy consumption for short code paths using rapl. *SIGMETRICS Perform. Eval. Rev.*, 40(3): 13–17, January 2012. ISSN 0163-5999. doi: 10.1145/2425248.2425252. URL https://doi.org/10.1145/2425248.2425252.

Hao, Z., Jiang, H., Jiang, S., Ren, J., and Cao, T. Hybrid slm and llm for edge-cloud collaborative inference. In *Proceedings of the Workshop on Edge and Mobile Foundation Models*, EdgeFM '24, pp. 36–41, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706639. doi: 10.1145/3662006.3662067. URL https://doi.org/10.1145/3662006.3662067.

Hockney, R. W. The communication challenge for mpp: Intel paragon and meiko cs-2. *Parallel Comput.*, 20:389–398, 1994. URL https://api.semanticscholar.org/CorpusID:22986998.

Holmes, C., Tanaka, M., Wyatt, M., Awan, A. A., Rasley, J., Rajbhandari, S., Aminabadi, R. Y., et al. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference, 2024. URL https://arxiv.org/abs/2401.08671.

Huang, K., Guo, X., and Wang, M. Specdec++: Boosting speculative decoding via adaptive candidate lengths, 2025. URL https://arxiv.org/abs/2405.19715.

Hubert, B. tc(8) — linux manual page. https://man7.org/linux/man-pages/man8/tc.8.html, 2001. Accessed: 2026-01-25.

Hugging Face. Gguf: A binary model file format for efficient loading and inference. https://huggingface.co/docs/hub/gguf, 2023.

Kim, S., Mangalam, K., Moon, S., Malik, J., Mahoney, M. W., Gholami, A., and Keutzer, K. Speculative decoding with big little decoder, 2023. URL https://arxiv.org/abs/2302.07863.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pp. 19274–19286, 2023.

Li, S., Wang, H., Xu, W., Zhang, R., Guo, S., Yuan, J., Zhong, X., Zhang, T., and Li, R. Collaborative inference and learning between edge slms and cloud llms: A survey of algorithms, execution, and open challenges, 2025a. URL https://arxiv.org/abs/2507.16731.

Li, Y., Wei, F., Zhang, C., and Zhang, H. Eagle: Speculative sampling requires rethinking feature uncertainty, 2025b. URL https://arxiv.org/abs/2401.15077.

Liu, J., Zeng, Q., Xu, H., Xu, Y., Wang, Z., and Huang, H. Adaptive block-wise regularization and knowledge distillation for enhancing federated learning. *IEEE/ACM Transactions on Networking*, 32(1):791–805, 2024. doi: 10.1109/TNET.2023.3301972.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, pp. 932–949. ACM, April 2024. doi: 10.1145/3620666.3651335. URL http://dx.doi.org/10.1145/3620666.3651335.

NVIDIA. TensorRT-LLM speculative decoding. https://nvidia.github.io/TensorRT-LLM/advanced/speculative-decoding.html, 2025. Accessed: 2026-01-11.

NVIDIA Corporation. *NVIDIA System Management Interface (nvidia-smi)*. NVIDIA Corporation, 2025. URL https://docs.nvidia.com/deploy/nvidia-smi/index.html. Accessed: 2026-01-28.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., et al. Gpt-4 technical report, 2024. URL https://arxiv.org/abs/2303.08774.

Park, J., Cho, S., and Han, D. Specedge: Scalable edge-assisted serving framework for interactive LLMs. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=4QVLKwgg3S.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., et al. Pytorch: An imperative style, high-performance deep learning library, 2019. URL https://arxiv.org/abs/1912.01703.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. 2019. URL https://api.semanticscholar.org/CorpusID:160025533.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Fu, D. Y., Xie, Z., et al. Flexgen: High-throughput generative inference of large language models with a single gpu, 2023. URL https://arxiv.org/abs/2303.06865.

Tian, Y., Zhang, Z., Yang, Y., Chen, Z., Yang, Z., Jin, R., Quek, T. Q. S., and Wong, K.-K. An edge-cloud collaboration framework for generative ai service provision with synergetic big cloud model and small edge models, 2024. URL https://arxiv.org/abs/2401.01666.

Tiwari, V., Malik, S., and Wolfe, A. Power analysis of embedded software: a first step towards software power minimization. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2(4):437–445, 1994. doi: 10.1109/92.335012.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., et al. Llama: Open and efficient foundation language models, 2023a. URL https://arxiv.org/abs/2302.13971.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., et al. Llama 2: Open foundation and fine-tuned chat models, 2023b. URL https://arxiv.org/abs/2307.09288.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pp. 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Wu, D., Ullah, R., Rodgers, P., Kilpatrick, P., Spence, I., and Varghese, B. Ecofed: Efficient communication for dnn partitioning-based federated learning. *IEEE Transactions on Parallel and Distributed Systems*, 35(3):377–390, 2024. doi: 10.1109/TPDS.2024.3349617.

Xie, Z., Xu, Y., Xu, H., Liao, Y., and Yao, Z. A novel hat-shaped device-cloud collaborative inference framework for large language models, 2025. URL https://arxiv.org/abs/2503.18989.

Xu, D., Yin, W., Zhang, H., Jin, X., Zhang, Y., Wei, S., Xu, M., and Liu, X. Edgellm: Fast on-device llm inference with speculative decoding. *IEEE Transactions on Mobile Computing*, 24(4):3256–3273, April 2025. ISSN 1536-1233. doi: 10.1109/TMC.2024.3513457. URL https://doi.org/10.1109/TMC.2024.3513457.

Zhan, H., Zhang, X., Tan, H., Tian, H., Yong, D., Zhang, J., and Li, X.-Y. Pice: A semantic-driven progressive inference system for llm serving in cloud-edge networks, 2025. URL https://arxiv.org/abs/2501.09367.

Zhang, P., Zeng, G., Wang, T., and Lu, W. Tinyllama: An open-source small language model, 2024. URL https://arxiv.org/abs/2401.02385.

Zhang, Z., Xu, J., Liang, T., Chen, X., He, Z., Wang, R., and Tu, Z. Draft model knows when to stop: Self-verification speculative decoding for long-form generation. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 16685–16697, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.844. URL https://aclanthology.org/2025.emnlp-main.844/.

Zhao, Y., Xie, Z., Liang, C., Zhuang, C., and Gu, J. Lookahead: An inference acceleration framework for large language model with lossless generation accuracy, 2024. URL https://arxiv.org/abs/2312.12728.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., et al. Sglang: Efficient execution of structured language model programs, 2024. URL https://arxiv.org/abs/2312.07104.

NVIDIA Corporation. *NVIDIA System Management Interface (nvidia-smi)*. NVIDIA Corporation, 2025. URL https://docs.nvidia.com/deploy/nvidia-smi/index.html. Accessed: 2026-01-28.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., et al. Gpt-4 technical report, 2024. URL https://arxiv.org/abs/2303.08774.

Park, J., Cho, S., and Han, D. Specedge: Scalable edge-assisted serving framework for interactive LLMs. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=4QVLKwgg3S.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., et al. Pytorch: An imperative style, high-performance deep learning library, 2019. URL https://arxiv.org/abs/1912.01703.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. 2019. URL https://api.semanticscholar.org/CorpusID:160025533.

Sheng, Y., Zheng, L., Yuan, B., Li, Z., Ryabinin, M., Fu, D. Y., Xie, Z., et al. Flexgen: High-throughput generative inference of large language models with a single gpu, 2023. URL https://arxiv.org/abs/2303.06865.

Tian, Y., Zhang, Z., Yang, Y., Chen, Z., Yang, Z., Jin, R., Quek, T. Q. S., and Wong, K.-K. An edge-cloud collaboration framework for generative ai service provision with synergetic big cloud model and small edge models, 2024. URL https://arxiv.org/abs/2401.01666.

Tiwari, V., Malik, S., and Wolfe, A. Power analysis of embedded software: a first step towards software power minimization. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2(4):437–445, 1994. doi: 10.1109/92.335012.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., et al. Llama: Open and efficient foundation language models, 2023a. URL https://arxiv.org/abs/2302.13971.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., et al. Llama 2: Open foundation and fine-tuned chat models, 2023b. URL https://arxiv.org/abs/2307.09288.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pp. 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Wu, D., Ullah, R., Rodgers, P., Kilpatrick, P., Spence, I., and Varghese, B. Ecofed: Efficient communication for dnn partitioning-based federated learning. *IEEE Transactions on Parallel and Distributed Systems*, 35(3):377–390, 2024. doi: 10.1109/TPDS.2024.3349617.

Xie, Z., Xu, Y., Xu, H., Liao, Y., and Yao, Z. A novel hat-shaped device-cloud collaborative inference framework for large language models, 2025. URL https://arxiv.org/abs/2503.18989.

Xu, D., Yin, W., Zhang, H., Jin, X., Zhang, Y., Wei, S., Xu, M., and Liu, X. Edgellm: Fast on-device llm inference with speculative decoding. *IEEE Transactions on Mobile Computing*, 24(4):3256–3273, April 2025. ISSN 1536-1233. doi: 10.1109/TMC.2024.3513457. URL https://doi.org/10.1109/TMC.2024.3513457.

Zhan, H., Zhang, X., Tan, H., Tian, H., Yong, D., Zhang, J., and Li, X.-Y. Pice: A semantic-driven progressive inference system for llm serving in cloud-edge networks, 2025. URL https://arxiv.org/abs/2501.09367.

Zhang, P., Zeng, G., Wang, T., and Lu, W. Tinyllama: An open-source small language model, 2024. URL https://arxiv.org/abs/2401.02385.

Zhang, Z., Xu, J., Liang, T., Chen, X., He, Z., Wang, R., and Tu, Z. Draft model knows when to stop: Self-verification speculative decoding for long-form generation. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 16685–16697, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.844. URL https://aclanthology.org/2025.emnlp-main.844/.

Zhao, Y., Xie, Z., Liang, C., Zhuang, C., and Gu, J. Lookahead: An inference acceleration framework for large language model with lossless generation accuracy, 2024. URL https://arxiv.org/abs/2312.12728.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., et al. Sglang: Efficient execution of structured language model programs, 2024. URL https://arxiv.org/abs/2312.07104.

Table A.1. Frequently used notations.

Symbol	Description
N	Number of draft tokens generated in one speculative round
$\hat{N}$	Token-batch scheduling window
K	Number of token batches in a batching strategy
$\mathbb{B}$	Token batching strategy, represented as a set of batch start indices
b_k	Index of the first draft token in the k-th batch
$t_c^{(k)}$	Cloud-edge communication time of the k-th token batch
$t_{ag}^{(k)}$	Autoregressive generation time of the k-th token batch
$\tau_{ag}^{(k)}$	Start time of autoregressive generation for the k-th batch
$\tau_c^{(k)}$	Start time of communication for the k-th batch
α	Cloud-edge communication startup overhead
β	Per-token transmission time
γ	Per-token autoregressive generation time on the edge device
T	Total generation and communication time of draft tokens in one speculative round
D_n	The n-th draft token generated by the draft model
$P(D_n)$	Confidence of draft token D_n
C_1	Cumulative sequence confidence of draft tokens
C_1^*	Updated cumulative sequence confidence after generating a new token
R_1	Threshold for cumulative sequence confidence triggering NAV
R_2	Threshold for single-token confidence triggering NAV
NAV	Non-autoregressive verification performed by the target model
TPT	Generation time per accepted token
ECS	Energy consumption of the cloud server per 100 accepted tokens

A. Modeling Rationale for Communication Time

In addition to empirical validation, we provide a theoretical rationale for our model. Specifically, the model is based on the widely adopted linear communication time model proposed by Hockney (Hockney, 1994). This model characterizes communication time using two key components: (1) Startup Overhead (α): This term accounts for the fixed latency incurred when initiating a communication session between the edge device and the cloud server (Gao et al., 2024). This overhead includes factors such as network handshake, connection establishment, and protocol negotiation, which are independent of the size of the data being transmitted. (2) Data Transmission Time ($\beta \cdot n$): This term represents the variable component of communication time that scales with the number of tokens (n) being transmitted in the batch. Here, β is the average time taken to transmit a single token over the network, which depends on factors such as bandwidth, network congestion, and packet size.

B. Proactive Transmission of Draft Tokens

To further reduce the edge-side idle time when waiting for verification result, PipeSD proactively generates and transmits draft tokens while verification is still in progress. Specifically, after sending the last batch of a speculative round, the edge immediately starts generating and transmitting the subsequent token batches without waiting for the verification result. The cloud buffers these proactively sent draft tokens upon arrival. After finishing NAV for the current round, the cloud checks whether (i) all draft tokens are accepted and (ii) the extra token generated by the target model matches the first buffered draft token. If both conditions hold, the buffered tokens remain valid and are kept; otherwise, they are discarded. Upon receiving the verification result, the edge applies the same check: if the draft tokens of the current round are fully accepted and the extra token given by the cloud matches the first proactively generated draft token, it resumes draft generation from where it left off; otherwise, it restarts draft generation from the last accepted token.

表 A.1. 常用符号。

Symbol	Description
N	Number of draft tokens generated in one speculative round
$\hat{N}$	Token-batch scheduling window
K	Number of token batches in a batching strategy
$\mathbb{B}$	Token batching strategy, represented as a set of batch start indices
b_k	Index of the first draft token in the k-th batch
$t_c^{(k)}$	Cloud-edge communication time of the k-th token batch
$t_{ag}^{(k)}$	Autoregressive generation time of the k-th token batch
$\tau_{ag}^{(k)}$	Start time of autoregressive generation for the k-th batch
$\tau_c^{(k)}$	Start time of communication for the k-th batch
α	Cloud-edge communication startup overhead
β	Per-token transmission time
γ	Per-token autoregressive generation time on the edge device
T	Total generation and communication time of draft tokens in one speculative round
D_n	The n-th draft token generated by the draft model
$P(D_n)$	Confidence of draft token D_n
C_1	Cumulative sequence confidence of draft tokens
C_1^*	Updated cumulative sequence confidence after generating a new token
R_1	Threshold for cumulative sequence confidence triggering NAV
R_2	Threshold for single-token confidence triggering NAV
NAV	Non-autoregressive verification performed by the target model
TPT	Generation time per accepted token
ECS	Energy consumption of the cloud server per 100 accepted tokens

A. 通信时间建模的原理

除了经验验证之外，我们还为我们的模型提供了理论依据。具体而言，该模型基于 Hockney 提出的广泛采用的线性通信时间模型。霍克尼 , 1994 ）。该模型使用两个关键组成部分来描述通信时间：(1) 启动开销（ α): 该术语指在启动边缘设备与云服务器之间的通信会话时产生的固定时延 Gaoet al., 2024 ）。这种开销包括网络握手、连接建立和协议协商等因素，这些因素与传输数据的大小无关。(2) 数据传输时间（ $\beta \cdot n$): 这个术语表示通信时间中随 token 数量而变化的可变部分 (n) 正在批量传输。这里， β 是传输单个 token 所需的平均时间，这取决于带宽、网络拥塞和数据包大小等因素。

B. 草稿代币的主动传输

为了进一步减少在等待验证结果时边缘端的空闲时间，PipeSD 在验证仍在进行时主动生成并传输草稿 token。具体来说，在发送完投机轮的最后一批后，边缘端会立即开始生成并传输后续的 token 批，而无需等待验证结果。云端在接收到这些主动发送的草稿 token 时会进行缓存。在完成当前轮的 NAV 后，云端会检查：(i) 是否所有草稿 token 都被接受，以及 (ii) 由目标模型生成的额外 token 是否与第一个缓存的草稿 token 匹配。如果两个条件都满足，缓存的 token 将保持有效并被保留；否则，它们将被丢弃。在接收到验证结果后，边缘端也会进行相同的检查：如果当前轮的草稿 token 是 如果完全被接受并且云提供的额外 token 与最初主动生成的草稿 token 匹配，它将从中断处继续生成草稿；否则，它将从最后被接受的 token 重新开始生成草稿。

C. Design and Performance Evaluation of BO

C.1. Parameter Settings and Advantage Analysis

We choose BO to obtain a near-optimal (R_1, R_2) benefiting from the following three points:

- First, BO is not limited by the expression of the objective function (we use $\mathcal{F}(R_1, R_2)$ as the objective function) and depends only on the sampling values obtained (i.e., $\hat{\mathcal{F}}(R_1^1, R_2^1), \hat{\mathcal{F}}(R_1^2, R_2^2), \ldots, \hat{\mathcal{F}}(R_1^n, R_2^n)$). We use Gaussian process regression with the Matern kernel to predict the value of the objective function as it is commonly used as a good surrogate model for BO (Alipourfard et al., 2017).
- Second, BO typically requires only a limited number of trials to find high-quality solutions, resulting in low search overhead. To minimize the number of trials, it selects the next configuration (R_1^1, R_2^1) by maximizing an acquisition function (Alipourfard et al., 2017). In PipeSD, we adopt the Expected Improvement (EI) acquisition function to choose the (R_1, R_2).
- Third, BO mitigates the risk of converging to a local optimum by adjusting the hyperparameter Expected Improvement (EI). Specifically, a smaller EI encourages exploitation by sampling more densely near the current optimum, whereas a larger EI promotes exploration by selecting more diverse points across the search space (Alipourfard et al., 2017). In PipeSD, we set EI $= 0.1$ to favor exploration over (R_1, R_2), which is a commonly adopted setting. Meanwhile, the search space of (R_1, R_2) in BO is defined as $(0, 1)^2$, and a single initial sample is randomly generated to initialize the optimization process.

C.2. Detailed Comparative Analysis of Tuning Strategies

To further justify the efficiency of the BO autotuner, we detail the implementation of the baseline search methods. In our evaluation, all methods share the same search space $(0, 1)^2$ for (R_1, R_2).

- **Grid Search:** The search space is discretized into a 4×4 uniform grid, resulting in 16 deterministic sampling points.
- **Random Search:** 16 pairs of (R_1, R_2) are sampled independently and uniformly from the continuous search space.

Evaluation Protocol: Each sampling point is evaluated by measuring the average TPT over 20 accepted tokens to ensure statistical stability and mitigate measurement noise. Table 3 shows the comparison of the average TPT when using different tuning methods on HumanEval and GSM8K. The results show that BO autotuner achieves the lowest TPT on both datasets. Although grid search obtains better performance than random number generation, its limited number of sampling points makes it difficult to cover the optimal solution especially when the search space is large, and over-increasing the number of sampling points brings higher search overhead and longer search process. On the contrary, BO can obtain a near-optimal solution with very little overhead, and therefore we choose it.

D. Robustness of PipeSD

In real-world deployment, the hardware environments including network bandwidth and computing power of the edge often change dynamically. In addition, task difficulty varies across inputs, which can shift the optimal NAV-triggering thresholds. To adapt to such dynamics, PipeSD continuously monitors online perfomance metrics and triggers targeted updates: (i) when the average TPT changes significantly, it re-runs the BO autotuner to update the NAV thresholds (R_1, R_2), and (ii) when the communication and computation parameters (α, β, γ) change significantly, it re-executes the DP scheduler to update the token batching strategy with updated parameters.

D.1. Automatic Update of NAV Thresholds

When average TPT changes noticeably, we re-run the BO autotuner to obtain new (R_1, R_2). We monitor TPT using a sliding window over the most recent 100 accepted tokens. An update is triggered only after the window is full and the relative change in TPT exceeds a predefined threshold δ_1:

$$\frac{|\text{TPT}_{\text{new}} - \text{TPT}_{\text{old}}|}{\text{TPT}_{\text{old}}} > \delta_1.$$

C. BO 的设计与性能评估

C.1. 参数设置与优势分析

我们选择 BO 来获得一个接近最优的 (R_1, R_2) 利益，基于以下三点：

- 首先，BO 不受目标函数表达式的限制（我们使用 $\mathcal{F}(R_1, R_2)$ 作为目标函数) 并且仅取决于所获得的采样值 (即 , $\hat{\mathcal{F}}(R_1^1, R_2^1)$ $\hat{\mathcal{F}}(R_1^2, R_2^2)$ $\ldots, \hat{\mathcal{F}}(R_1^n, R_2^n)$)。我们使用带有 Matern 核的高斯过程回归来预测目标函数的值，因为它通常被用作 BO 的优秀代理模型 (Alipourfard et al.,2017).
- 其次，BO 通常只需要有限次数的试验就能找到高质量的解决方案，从而造成较低的搜索开销。为了最小化试验次数，它会选择下一个配置 (R_1^1, R_2^1) 通过最大化一个采集函数 (Alipourfard etal., 2017 在 PipeSD 中，我们采用期望改进（ EI ）采集函数来选择 (R_1, R_2).
- 第三，BO 通过调整超参数期望改进（ EI ）来减轻收敛到局部最优的风险。具体而言，较小的 EI 通过在当前最优附近进行更密集的采样来鼓励利用，而较大的 EI 通过在搜索空间中选择更多样的点来促进探索。Alipourfard et al., 2017). 在 PipeSD 中，我们设置 EI $= 0.1$ 偏向探索而非 $(R_1, R_2$ ），这是一个常用的设置。与此同时，（的搜索空间 R_1, R_2) 在 BO 中定义为 (0,1)₂，并且随机生成一个初始样本来初始化优化过程。

C.2. 调优策略的详细比较分析

为了进一步证明 BO 自动调优器的效率，我们详细说明了基线搜索方法的实现。在我们的评估中，所有方法共享相同的搜索空间 $(0,1)^2$ 对于 (R_1, R_2)。

- **网格搜索：**海洋　　搜索空间被离散为一个 4×4 均匀网格，从而得到 16 个确定性的采样点。
- **随机搜索：**从连续搜索空间中独立、均匀地采样 16 对（ R_1, R_2 ）。

评估协议：每个采样点通过测量 20 个接受 token 的平均 TPT 来进行评估，以确保统计稳定性并减轻测量噪声。表 3 显示了在 HumanEval 和 GSM8K 上使用不同调优方法时平均 TPT 的比较。结果表明，BO 自动调优器在两个数据集上均实现了最低的 TPT。尽管网格搜索的性能优于随机数生成，但由于采样点数量有限，很难覆盖最优解，尤其是在搜索空间较大时，并且采样点数量过多会带来更高的搜索开销和更长的搜索过程。相反，BO 可以以极少的开销获得接近最优的解，因此我们选择它。

D. PipeSD 的稳健性

在实际部署中，边缘设备的硬件环境，包括网络带宽和计算能力，往往会动态变化。此外，任务难度在不同输入之间也会有所不同，这可能会改变最优的 NAV 触发阈值。为了适应这种动态变化，PipeSD 持续监控在线性能指标并触发针对性更新：（ i ）当平均 TPT 发生显著变化时，它重新运行 BO 自动调优器以更新 NAV 阈值（ R_1, R_2 ）；（ ii ）当通信和计算参数（ α, β, γ ）发生显著变化时，它重新执行 DP 调度器以使用更新的参数更新 token 批处理策略。

D.1. NAV 阈值的自动更新

当平均 TPT 明显变化时，我们会重新运行 BO 自动调整器以获得新的 (R_1, R_2)。我们使用最近 100 个被接受的 token 的滑动窗口来监控 TPT。仅在窗口填满且 TPT 的相对变化超过预设阈值时才触发更新 δ_1:

$$\frac{|\text{TPT}_{\text{new}} - \text{TPT}_{\text{old}}|}{\text{TPT}_{\text{old}}} > \delta_1.$$

Here, TPT_{new} and TPT_{old} denote the average TPTs of the current and previous windows, respectively. The threshold δ_1 depends on the hardware environment and inference configuration. When the condition is met, the BO autotuner is re-executed asynchronously.

D.2. Automatic Update of Token Batching Strategy

When the communication and computation parameters (α, β, and γ) change significantly, we update the token batching strategy by re-running the DP scheduler (Algorithm 1) with updated parameters. We estimate these parameters using a sliding window over the most recent 100 transmitted token batches, as described below.

- **Estimating γ.** We compute γ as the average *per-token* generation time over the most recent 100 generated batches (or all available batches if fewer than 100 are available).
- **Estimating α and β.** We bootstrap the estimation by transmitting 8 token batches with sizes 1–8 and recording their end-to-end communication times, which provides the initial data points for regression. Afterward, we maintain a sliding window over the most recent 100 transmitted batches; the estimation below is performed only once the window is full. For each batch in the window, we record its end-to-end communication time, group batches by size, and compute the average communication time per size. If fewer than 8 distinct batch sizes appear in the window, we proactively transmit additional batches with previously unseen sizes (starting from the smallest unseen sizes) to obtain up to 8 data points. We then fit a linear model of average communication time versus batch size, where the intercept and slope correspond to α and β, respectively.

To avoid unnecessary re-scheduling, we trigger a DP update only when the estimated parameters change noticeably. Specifically, we re-run the DP scheduler if the relative change in γ exceeds δ_2, or if the relative change in either α or β exceeds a shared threshold δ_3:

$$\frac{|\gamma_{\text{new}} - \gamma_{\text{old}}|}{\gamma_{\text{old}}} > \delta_2 \quad \text{or} \quad \frac{|\alpha_{\text{new}} - \alpha_{\text{old}}|}{\alpha_{\text{old}}} > \delta_3 \quad \text{or} \quad \frac{|\beta_{\text{new}} - \beta_{\text{old}}|}{\beta_{\text{old}}} > \delta_3.$$

Here, $(\alpha_{\text{new}}, \beta_{\text{new}}, \gamma_{\text{new}})$ and $(\alpha_{\text{old}}, \beta_{\text{old}}, \gamma_{\text{old}})$ are the parameter estimates from the current and previous windows, respectively. The thresholds δ_2 and δ_3 depend on the hardware environment and inference configuration. When either condition holds, we update the token batching strategy by re-executing the DP scheduler.

If the thresholds are set too small, measurement noise may frequently trigger unnecessary updates, resulting in excessive overhead. In contrast, overly large thresholds may delay the detection of meaningful environmental changes, reducing the adaptivity of PipeSD.

In our experiments, we empirically set

$$\delta_1 = \delta_2 = \delta_3 = 0.2,$$

which provides a good balance between adaptivity and update overhead. We further observe that PipeSD is not highly sensitive to the exact threshold values within a moderate range. Specifically, when the thresholds vary within $[0.1, 0.5]$, the overall inference performance remains very similar across all evaluated scenarios. This empirical observation demonstrates the robustness of PipeSD with respect to threshold selection.

E. Proof of Theorem 4.1

For any $j \in \{0, 1, \ldots, \hat{N}\}$, define $\text{OPT}(j)$ as the minimum possible autoregressive generation and communication completion time of all tokens $\{1, \ldots, j\}$. Clearly, $\text{OPT}(0) = 0$.

Consider an optimal strategy for the prefix $\{1, \ldots, j\}$ with $j \geq 1$. Let the last batch be $\{i+1, \ldots, j\}$ for some $i \in \{0, \ldots, j-1\}$. Its communication time is $\alpha + \beta\,(j - i)$. The communication of this last batch can start only after both the previous batch has completed communication and the current batch has completed autoregressive generation. The former completes at time $\text{OPT}(i)$ by definition. Our experimental results (Figure 6b) show that the per-token generation time on the edge device remains approximately constant with respect to the prefix length. We denote this constant generation time by γ. Consequently, generating j tokens requires approximately γj time, and the latter condition is satisfied at time γj.

这里，TPT_{new} 和 TPT_{old} 分别表示当前窗口和前一个窗口的平均 TPT。阈值 δ_1 取决于硬件环境和推理配置。当条件满足时，BO 自动调优器会异步重新执行。

D.2. token 批处理策略的自动更新

当通信与计算参数（ α, β 和 γ ）显著变化时，我们使用更新后的参数重新运行 DP 调度器（算法 1 ），以更新 token 批次策略。如下所述，我们使用最近 100 个已传输 token 批次组成的滑动窗口估计这些参数。

- **估计** γ。 我们计算 γ 作为最近 100 个生成批次（如果可用批次少于 100，则为所有可用批次）每个 token 生成时间的平均值 。
- **估算** α **和** β。我们通过传输大小为 1–8 的 8 个 token 批次并记录其端到端通信时间来启动估算，这为回归提供了初始数据点。之后，我们保持对最近传输的 100 个批次的滑动窗口；以下估算仅在窗口填满后执行。对于窗口中的每个批次，我们记录其端到端通信时间，按批次大小对批次进行分组，并计算每个大小的平均通信时间。如果窗口中出现的不同批次大小少于 8 种，我们会主动传输先前未见过的新大小批次（从最小的未见大小开始）以获得最多 8 个数据点。然后我们拟合平均值的线性模型 通信时间与批量大小的关系，其中截距和斜率分别对应于 α 和 β。

为了避免不必要的重新调度，我们仅在估计参数明显变化时触发 DP 更新。具体而言，如果 γ 的相对变化超过 δ_2，或者 α 或 β 的相对变化超过共享阈值 δ_3，则我们会重新运行 DP 调度器 :

$$\frac{|\gamma_{\text{new}} - \gamma_{\text{old}}|}{\gamma_{\text{old}}} > \delta_2 \quad \text{or} \quad \frac{|\alpha_{\text{new}} - \alpha_{\text{old}}|}{\alpha_{\text{old}}} > \delta_3 \quad \text{or} \quad \frac{|\beta_{\text{new}} - \beta_{\text{old}}|}{\beta_{\text{old}}} > \delta_3.$$

这里，$(\alpha_{\text{new}}, \beta_{\text{new}}, \gamma_{\text{new}})$ 和 $(\alpha_{\text{old}}, \beta_{\text{old}}, \gamma_{\text{old}})$ 分别是当前和前一个窗口的参数估计值。阈值 δ_2 和 δ_3 取决于硬件环境和推理配置。当任一条件成立时，我们通过重新执行 DP 调度器来更新 token 批处理策略。

如果阈值设置得过小，测量噪声可能会频繁触发不必要的更新，从而导致过多的开销。相反，阈值过大可能会时延对有意义的环境变化的检测，从而降低 PipeSD 的适应性。

在我们的实验中，我们通过经验设定

$$\delta_1 = \delta_2 = \delta_3 = 0.2,$$

它在适应性和更新开销之间提供了良好的平衡。我们进一步观察到，PipeSD 对于适度范围内的具体阈值并不高度敏感。具体来说，当阈值在 $[0.1, 0.5]$ 范围内变化时，在所有评估的场景中整体推理性能保持非常相似。这一经验性观察表明 PipeSD 对阈值选择具有很强的鲁棒性。

E。定理 4.1 的证明

对于任意 $j \in \{0, 1, \ldots, \hat{N}\}$，记 $\text{OPT}(j)$ 为所有 token $\{1, \ldots, j\}$ 的最小可能自回归生成和通信完成时间。显然，$\text{OPT}(0) = 0$。

考虑一个关于预 x 的最优策略 $\{1, \ldots, j\}$ 与 $j \geq 1$ 让最后一批 $\{i+1, \ldots, j\}$ 对于一些 $i \in \{0, \ldots, j-1\}$. 它的通信时间是 $\alpha + \beta\,(j - i)$. 只有在前一批次完成通信且当前批次完成自回归生成之后，本批次的通信才能开始。前者在时间完成 $\text{OPT}(i)$ 按定义。我们的实验结果（图 6b ）表明边缘设备上的每个 token 生成时间相对于前缀长度保持大致不变。我们将这一恒定的生成时间表示为 γ 因此，生成 j 大约需要代币 γj 时间，并且后一个条件在时间上得到满足 γj.

Table A.2. Speedup of DP-based token batching over stronger pipelined baselines under different communication settings.

(α, β) (ms)	DP vs Greedy	DP vs Immediate-send	DP vs No-early-upload
$(20, 72)$	$1.02\times$	$1.09\times$	$1.22\times$
$(100, 72)$	$1.04\times$	$1.44\times$	$1.10\times$
$(200, 72)$	$1.10\times$	$1.76\times$	$1.06\times$
$(20, 48)$	$1.03\times$	$1.11\times$	$1.33\times$
$(100, 48)$	$1.06\times$	$1.63\times$	$1.13\times$
$(200, 48)$	$1.13\times$	$2.06\times$	$1.06\times$

Therefore, the earliest possible communication start time of the last batch is

$$\max\{\mathrm{OPT}(i),\, \gamma j\},$$

and the resulting completion time equals

$$\max\{\mathrm{OPT}(i),\, \gamma j\} + \alpha + \beta\,(j - i).$$

Minimizing over all possible i yields the following recurrence:

$$\mathrm{OPT}(j) = \min_{0 \le i \le j-1} \left(\max\{\mathrm{OPT}(i),\, \gamma j\} + \alpha + \beta\,(j - i)\right). \tag{7}$$

The DP in Algorithm 1 computes $\mathrm{dp}[j]$ using exactly the recurrence in (7) with base $\mathrm{dp}[0] = 0$, hence by induction on j we have $\mathrm{dp}[j] = \mathrm{OPT}(j)$ for all $j \le \hat{N}$. In particular, $\mathrm{dp}[\hat{N}] = \mathrm{OPT}(\hat{N})$, so the minimum completion time is achieved.

Finally, storing an predecessor for each j and backtracking reconstructs a partition whose boundaries attain the minimum in (7) at every step, and thus yields an optimal batching strategy $\mathbb{B}$.

Therefore, Algorithm 1 returns an optimal token batching strategy.

F. Necessity of DP-based Token Batching

To further justify the necessity of the DP-based token-batching policy, we compare PipeSD with several stronger pipelined baselines beyond the no-pipeline ablation in Table 6. The goal of this comparison is to isolate whether the gain of PipeSD comes merely from introducing pipelining, or from the DP-based optimization of token-batch transmission itself.

We consider the following baselines:

- **Greedy:** whenever the network becomes idle, the edge immediately transmits all currently accumulated draft tokens.
- **Immediate-send:** each draft token is transmitted as soon as it is generated.
- **No-early-upload:** the edge first generates the whole draft sequence and then uploads it to the cloud, i.e., no transmission pipelining is applied.

We evaluate these methods under different communication settings characterized by the startup latency α and per-token transmission latency β. Table A.2 reports the speedup of DP over the above baselines.

The results show that DP consistently outperforms all three alternatives. In particular, compared with the greedy policy, DP still achieves $1.02\times$–$1.13\times$ speedup across all tested settings. This indicates that the advantage of PipeSD does not merely come from pipelining itself, but from optimizing *when* and *how many* tokens should be transmitted under communication startup overhead. The gain over immediate-send is even larger, especially when α is high, because transmitting too frequently incurs excessive startup cost. Meanwhile, DP also consistently outperforms no-early-upload, confirming the benefit of overlapping token generation with communication.

These results, together with the negligible DP overhead reported in Table 5, demonstrate that the DP scheduler is both necessary and practical in PipeSD.

Table A.2. 在不同通信设置下，基于 DP 的 token 批次划分相对于更强流水线基线的加速比。

(α, β) (ms)	DP vs Greedy	DP vs Immediate-send	DP vs No-early-upload
$(20, 72)$	$1.02\times$	$1.09\times$	$1.22\times$
$(100, 72)$	$1.04\times$	$1.44\times$	$1.10\times$
$(200, 72)$	$1.10\times$	$1.76\times$	$1.06\times$
$(20, 48)$	$1.03\times$	$1.11\times$	$1.33\times$
$(100, 48)$	$1.06\times$	$1.63\times$	$1.13\times$
$(200, 48)$	$1.13\times$	$2.06\times$	$1.06\times$

因此，最后一批的最早可能通信开始时间是

$$\max\{\mathrm{OPT}(i),\, \gamma j\},$$

而由此产生的完成时间等于

$$\max\{\mathrm{OPT}(i),\, \gamma j\} + \alpha + \beta\,(j - i).$$

对所有可能的 i 求最小值得到以下递推关系：

$$\mathrm{OPT}(j) = \min_{0 \le i \le j-1} \left(\max\{\mathrm{OPT}(i),\, \gamma j\} + \alpha + \beta\,(j - i)\right). \tag{7}$$

算法 1 中的 DP 计算 $\mathrm{dp}[j]$，完全使用公式 (7) 中的递推关系，以基础 $\mathrm{dp}[0] = 0$ 为起点，因此通过对 j 的归纳，我们得到 $\mathrm{dp}[j] = \mathrm{OPT}(j)$ 对所有 $j \le \hat{N}$ 均成立。特别地，$\mathrm{dp}[\hat{N}] = \mathrm{OPT}(\hat{N})$，因此达到了最小完成时间。

最后，为每个 j 存储一个前驱，并通过回溯重建一个其边界在每一步都达到 (7) 中最小值的划分，从而得到一个最优的批处理策略 $\mathbb{B}$。

因此，算法 1 返回最优 token 批次策略。

F. 基于 DP 的 token 批处理的必要性

为进一步验证基于 DP 的 token 批次策略的必要性，除表 6 中关闭流水线的消融配置外，我们还将 PipeSD 与若干更强的流水线基线比较。该比较旨在区分：PipeSD 的收益是否仅来自引入流水线，还是来自对 token 批次传输本身进行 DP 优化。

我们考虑以下基线：

- **贪婪：**每当　　当网络变为空闲时，边缘立即传输当前积累的全部草稿 token。
- **即时发送：**每个草稿 token 在生成后立即传输。
- **无早期上传：**边缘首先生成整个草稿序列，然后将其上传到云端，即不应用传输流水线。

我们在不同的通信设置下评估这些方法，这些设置的特征是启动时延 α 和每个 token 的传输时延 β。表 A.2 报告了 DP 相对于上述基线的加速情况。

结果显示，DP 始终优于其他三种替代方法。特别是，与贪心策略相比，DP 在所有测试设置下仍实现了 $1.02\times$–$1.13\times$ 的加速。这表明 PipeSD 的优势不仅仅来自流水线本身，而是来自于在通信启动开销下优化 何时以及 多少 token 应当被传输。相比立即发送的收益甚至更大，尤其是在 α 较高时，因为过于频繁的传输会产生过高的启动成本。与此同时，DP 也始终优于不提前上传，确认了将 token 生成与通信重叠的好处。

这些结果与表 5 中可忽略的 DP 开销共同表明，DP 调度器在 PipeSD 中既有必要，也具备实用性。

G. Details of Experimental Setup

G.1. Network Bandwidth Control

To ensure reproducible and fair comparisons, we conduct all experiments under fixed network bandwidth settings, except where explicitly noted. We limit the bandwidth on the cloud-edge link using OS-specific traffic-control tools on each endpoint. The edge device runs Windows 11; we limit the uplink bandwidth using Windows Policy-based QoS by configuring an outbound throttling rate for the traffic from the edge to the cloud. The cloud server runs Ubuntu 22.04; we limit the downlink bandwidth using Linux Traffic Control (`tc`) (Hubert, 2001).

G.2. Edge Compute Emulation with Artificial Delays

To emulate Scenario 2 and Scenario 3 with limited computing power of the edge device, we inject artificial delays into token generation on the edge device. Our physical edge device (Lenovo ThinkBook 16+) has a CPU frequency of 5.1 GHz. We simulate lower CPU frequencies of 2.5 GHz (Scenario 2, smartphone-class) and 1.2 GHz (Scenario 3, IoT-class) by adding an extra delay after each generated token. The per-token artificial delay is computed as

$$\text{Artificial Delay} = \text{Base Generation Time} \times \left(\frac{\text{Real CPU Frequency}}{\text{Simulated CPU Frequency}} - 1 \right),$$

where Base Generation Time denotes the time to generate one token on the physical edge device.

G.3. EdgeLLM Adaptation Details

EdgeLLM is originally designed for edge-only inference scenarios, but its two core ideas transfer naturally to our cloud-edge collaborative setting: (i) continuing draft generation while waiting for NAV, and (ii) triggering NAV when the cumulative sequence confidence falls below a dynamically adjusted cumulative sequence confidence threshold R_1.

The first technique is implemented in the same way as our proactive transmission of draft tokens (Appendix B). Below we describe EdgeLLM's dynamic threshold mechanism. Consider a speculative round with a scheduling window of $\hat{N}$ draft tokens. After each NAV, we update R_1 using the following rule:

$$R_{1,\text{new}} = \begin{cases} 0.5\, R_1, & \text{if } N_{\text{correct}} = \hat{N}, \\ \dfrac{R_1}{C_1^{\frac{\hat{N}-N_{\text{correct}}}{\hat{N}}}}, & \text{if } N_{\text{correct}} < \hat{N}. \end{cases} \tag{7}$$

where N_{correct} is the number of accepted tokens in the current round, and C_1 is the cumulative sequence confidence before NAV. In our experiments, we choose the optimal initial value of R_1 for each experimental setup.

H. Edge Energy Overhead

We analyze the additional energy consumed on the edge due to PipeSD's control-plane logic (BO autotuning, DP scheduling, and online parameter measurement). Let P_{idle} denote the CPU idle power on the edge device. When the edge executes a workload x, its average power is modeled as $P_{\text{idle}} + P_x$, where P_x is the workload-induced incremental power above idle (Hähnel et al., 2012; Tiwari et al., 1994). In particular, let P_{ag} be the incremental power of autoregressive token generation, and let P_{BO}, P_{DP}, and P_{PM} be the incremental powers of BO, DP, and parameter measurement, respectively. Let T_{BO}, T_{DP}, and T_{PM} be their average per-round runtimes, and let T be the average duration of one speculative decoding round. Note that BO autotuning, DP scheduling, and parameter measurement are *not* executed in every speculative round; they are triggered only when necessary. Therefore, in the per-round analysis, the terms T_{BO}, T_{DP}, and T_{PM} should be interpreted as *amortized* runtimes, i.e., the total time spent on these routines over many rounds divided by the number of rounds.

We consider a conservative (worst-case) setting where these control-plane steps are *not* overlapped with draft generation. Then the edge spends $T - T_{\text{BO}} - T_{\text{DP}} - T_{\text{PM}}$ on draft generation. The total edge energy per round can be written as

$$\begin{aligned} W \;&= (T - T_{\text{BO}} - T_{\text{DP}} - T_{\text{PM}})\,(P_{\text{ag}} + P_{\text{idle}}) \\ &\quad + T_{\text{DP}}(P_{\text{DP}} + P_{\text{idle}}) + T_{\text{PM}}(P_{\text{PM}} + P_{\text{idle}}) + T_{\text{BO}}(P_{\text{BO}} + P_{\text{idle}}). \end{aligned} \tag{8}$$

G. 实验设置的详细信息

G.1. 网络带宽控制

: 为了确保可重复和公平的比较，除非另有明确说明，我们在固定的网络带宽设置下进行所有实验。我们使用每个端点的操作系统特定流量控制工具限制云边链路上的带宽。边缘设备运行 Windows 11； 我们通过配置从边缘到云的流量的出站限速来使用 Windows 基于策略的 QoS 限制上行带宽。云服务器运行 Ubuntu 22.04； 我们使用 Linux 流量控制 (`tc`) (Hubert,2001).

G.2. 带人工时延的边缘计算仿真

为了在边缘设备有限的计算能力下模拟场景 2 和场景 3，我们在边缘设备上向 token 生成中注入人工时延。我们的物理边缘设备（联想 ThinkBook 16+） CPU 频率为 5.1 GHz。我们通过在每生成一个 token 后添加额外时延来模拟较低的 CPU 频率，分别为 2.5 GHz （场景 2，智能手机级）和 1.2 GHz （场景 3，物联网级）。每个 token 的人工时延计算如下

$$\text{Artificial Delay} = \text{Base Generation Time} \times \left(\frac{\text{Real CPU Frequency}}{\text{Simulated CPU Frequency}} - 1 \right),$$

w这里的基础生成时间表示在物理边缘设备上生成一个 token 所需的时间 .

G.3. EdgeLLM 适配详情

EdgeLLM 最初是为仅边缘推理场景设计的，但它的两个核心理念可以自然地转移到我们的云 - 边缘协作环境中：(i) 在等待 NAV 时继续生成草稿，(ii) 当累积序列置信度低于动态调整的累积序列置信度阈值 R_1 时触发 NAV。

第一种技术的实现方式与我们主动传输草稿 token 的方式相同（附录 B ）。下面我们描述 EdgeLLM 的动态阈值机制。考虑一个具有 $\hat{N}$ 草稿 token 调度窗口的推测回合。在每次 NAV 之后，我们使用以下规则更新 R_1：

$$R_{1,\text{new}} = \begin{cases} 0.5\, R_1, & \text{if } N_{\text{correct}} = \hat{N}, \\ \dfrac{R_1}{C_1^{\frac{\hat{N}-N_{\text{correct}}}{\hat{N}}}}, & \text{if } N_{\text{correct}} < \hat{N}. \end{cases} \tag{7}$$

其中 N_{correct} 是当前回合中接受的 token 数量，C_1 是 NAV 之前的累积序列置信度。在我们的实验中，我们为每个实验设置选择 R_1 的最优初始值。

H. 边缘能量开销

我们分析了由于 PipeSD 的控制平面逻辑（ BO 自动调优、 DP 调度和在线参数测量）在边缘消耗的额外能量。设 P_{idle} 表示边缘设备的 CPU 空闲功耗。当边缘执行工作负载 x 时，其平均功耗被建模为 $P_{\text{idle}} + P_x$，其中 P_x 是高于空闲状态的工作负载引起的增量功耗（ H¨ ahnel 等 , 2012； Tiwari 等 , 1994 ）。具体地，设 P_{ag} 为自回归 token 生成的增量功耗，设 P_{BO}、 P_{DP} 和 P_{PM} 分别为 BO 、 DP 和参数测量的增量功耗。设 T_{BO}、 T_{DP} 和 T_{PM} 为它们每轮的平均运行时间，且 让 T 表示一次推测解码轮次的平均持续时间。注意，BO 自动调优、 DP 调度和参数测量并不是在每一轮推测中执行的；它们仅在必要时触发。因此，在每轮分析中，术语 T_{BO}、 T_{DP} 和 T_{PM} 应被解释为摊销运行时间，即在多轮中这些例程消耗的总时间除以轮次数。

我们考虑一种保守的最坏情况：这些控制面步骤不与草稿生成重叠。此时，边缘执行草稿生成所需的时间为 $T - T_{\text{BO}} - T_{\text{DP}} - T_{\text{PM}}$。每轮总边缘能耗可写为

$$\begin{aligned} W \;&= (T - T_{\text{BO}} - T_{\text{DP}} - T_{\text{PM}})\,(P_{\text{ag}} + P_{\text{idle}}) \\ &\quad + T_{\text{DP}}(P_{\text{DP}} + P_{\text{idle}}) + T_{\text{PM}}(P_{\text{PM}} + P_{\text{idle}}) + T_{\text{BO}}(P_{\text{BO}} + P_{\text{idle}}). \end{aligned} \tag{8}$$

The energy spent on the control-plane routines is

$$\Delta W \;=\; T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}}), \tag{9}$$

and the fraction of control-plane energy is

$$\frac{\Delta W}{W}=\frac{T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}})}{(T-T_{\mathrm{BO}}-T_{\mathrm{DP}}-T_{\mathrm{PM}})\,(P_{\mathrm{ag}}+P_{\mathrm{idle}})+T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}})}. \tag{10}$$

Since BO/DP/measurement are lightweight compared to draft decoding, their incremental powers are typically smaller, i.e., $P_{\mathrm{BO}}<P_{\mathrm{ag}}$, $P_{\mathrm{DP}}<P_{\mathrm{ag}}$, and $P_{\mathrm{PM}}<P_{\mathrm{ag}}$. Equivalently,

$$P_{\mathrm{BO}}+P_{\mathrm{idle}}\le P_{\mathrm{ag}}+P_{\mathrm{idle}},\quad P_{\mathrm{DP}}+P_{\mathrm{idle}}\le P_{\mathrm{ag}}+P_{\mathrm{idle}},\quad P_{\mathrm{PM}}+P_{\mathrm{idle}}\le P_{\mathrm{ag}}+P_{\mathrm{idle}}. \tag{11}$$

Let $S\triangleq T_{\mathrm{BO}}+T_{\mathrm{DP}}+T_{\mathrm{PM}}$. From (10), we can rewrite

$$\frac{\Delta W}{W}=\frac{\Delta W}{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})+\Delta W}=\frac{1}{1+\frac{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})}{\Delta W}}. \tag{12}$$

Moreover, by (11), the control-plane energy satisfies

$$\begin{aligned}\Delta W&=T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}})\\&\le(T_{\mathrm{DP}}+T_{\mathrm{PM}}+T_{\mathrm{BO}})\,(P_{\mathrm{ag}}+P_{\mathrm{idle}})=S\,(P_{\mathrm{ag}}+P_{\mathrm{idle}}).\end{aligned} \tag{13}$$

Plugging (13) into (12) yields

$$\frac{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})}{\Delta W}\ge\frac{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})}{S(P_{\mathrm{ag}}+P_{\mathrm{idle}})}=\frac{T-S}{S}, \tag{14}$$

and thus

$$\frac{\Delta W}{W}\le\frac{1}{1+\frac{T-S}{S}}=\frac{S}{T}=\frac{T_{\mathrm{BO}}+T_{\mathrm{DP}}+T_{\mathrm{PM}}}{T}. \tag{15}$$

Instantiating the bound with our measurements. To connect the bound to our experimental setup, consider the first $R=1000$ speculative rounds in Scenario 1. Let T_{tot} be the total wall-clock time of these R rounds, and let $T_{\mathrm{BO,tot}}$, $T_{\mathrm{DP,tot}}$, and $T_{\mathrm{PM,tot}}$ be the total runtimes spent on BO, DP, and parameter measurement within the same period, respectively. Over $R=1000$ speculative rounds, the control-plane energy fraction satisfies

$$\frac{\Delta W_{\mathrm{tot}}}{W_{\mathrm{tot}}}=\frac{R\,\Delta W}{R\,W}\le\frac{R\,S}{R\,T}=\frac{T_{\mathrm{BO,tot}}+T_{\mathrm{DP,tot}}+T_{\mathrm{PM,tot}}}{T_{\mathrm{tot}}}=\frac{T_{\mathrm{BO,tot}}}{T_{\mathrm{tot}}}+\frac{T_{\mathrm{DP,tot}}}{T_{\mathrm{tot}}}+\frac{T_{\mathrm{PM,tot}}}{T_{\mathrm{tot}}}. \tag{16}$$

where ΔW_{tot} and W_{tot} denote the total control-plane energy and total energy over the R rounds, respectively. Using the profiling results in Table 5, we have

$$\frac{T_{\mathrm{BO,tot}}}{T_{\mathrm{tot}}}\le 1.1\%,\quad \frac{T_{\mathrm{DP,tot}}}{T_{\mathrm{tot}}}\le 0.013\%,\quad \frac{T_{\mathrm{PM,tot}}}{T_{\mathrm{tot}}}\le 0.4\%.$$

Therefore, the control-plane energy fraction is bounded by

$$\frac{\Delta W_{\mathrm{tot}}}{W_{\mathrm{tot}}}\le 1.1\%+0.013\%+0.4\%=1.513\%.$$

This indicates that the additional energy consumption on the edge due to PipeSD's control-plane logic is at most 1.513% of the total edge energy consumption in our experiments, which is negligible.

用于控制平面例程的能量是

$$\Delta W \;=\; T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}}), \tag{9}$$

以及控制平面能量的分数是

$$\frac{\Delta W}{W}=\frac{T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}})}{(T-T_{\mathrm{BO}}-T_{\mathrm{DP}}-T_{\mathrm{PM}})\,(P_{\mathrm{ag}}+P_{\mathrm{idle}})+T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}})}. \tag{10}$$

由于 BO/DP/ 测量 相对于草稿解码来说是轻量级的，它们的增量能力通常较小，即 $P_{\mathrm{BO}}<P_{\mathrm{ag}}$、 $P_{\mathrm{DP}}<P_{\mathrm{ag}}$ 和 $P_{\mathrm{PM}}<P_{\mathrm{ag}}$。等价地，

$$P_{\mathrm{BO}}+P_{\mathrm{idle}}\le P_{\mathrm{ag}}+P_{\mathrm{idle}},\quad P_{\mathrm{DP}}+P_{\mathrm{idle}}\le P_{\mathrm{ag}}+P_{\mathrm{idle}},\quad P_{\mathrm{PM}}+P_{\mathrm{idle}}\le P_{\mathrm{ag}}+P_{\mathrm{idle}}. \tag{11}$$

Let $S\triangleq T_{\mathrm{BO}}+T_{\mathrm{DP}}+T_{\mathrm{PM}}$。 从 (10)，我们可以重写

$$\frac{\Delta W}{W}=\frac{\Delta W}{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})+\Delta W}=\frac{1}{1+\frac{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})}{\Delta W}}. \tag{12}$$

此外，根据式（ 11 ），控制面能耗满足

$$\begin{aligned}\Delta W&=T_{\mathrm{DP}}(P_{\mathrm{DP}}+P_{\mathrm{idle}})+T_{\mathrm{PM}}(P_{\mathrm{PM}}+P_{\mathrm{idle}})+T_{\mathrm{BO}}(P_{\mathrm{BO}}+P_{\mathrm{idle}})\\&\le(T_{\mathrm{DP}}+T_{\mathrm{PM}}+T_{\mathrm{BO}})\,(P_{\mathrm{ag}}+P_{\mathrm{idle}})=S\,(P_{\mathrm{ag}}+P_{\mathrm{idle}}).\end{aligned} \tag{13}$$

将 (Plugging (13)) 插入 (12) 得到

$$\frac{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})}{\Delta W}\ge\frac{(T-S)(P_{\mathrm{ag}}+P_{\mathrm{idle}})}{S(P_{\mathrm{ag}}+P_{\mathrm{idle}})}=\frac{T-S}{S}, \tag{14}$$

因此

$$\frac{\Delta W}{W}\le\frac{1}{1+\frac{T-S}{S}}=\frac{S}{T}=\frac{T_{\mathrm{BO}}+T_{\mathrm{DP}}+T_{\mathrm{PM}}}{T}. \tag{15}$$

用我们的测量值具体化该界限。 为了将边界连接到我们的实验装置，请考虑第一个 $R=1000$ 情景 1 中的推测轮次。让 T_{tot} 成为这些的总挂钟时间 R 回合，并且让 $T_{\mathrm{BO,tot}}$, $T_{\mathrm{DP,tot}}$，和 $T_{\mathrm{PM,tot}}$ 分别是在同一时期内在 BO 、DP 和参数测量上花费的总运行时间。过 $R=1000$ 投机回合，控制平面能量分数满足

$$\frac{\Delta W_{\mathrm{tot}}}{W_{\mathrm{tot}}}=\frac{R\,\Delta W}{R\,W}\le\frac{R\,S}{R\,T}=\frac{T_{\mathrm{BO,tot}}+T_{\mathrm{DP,tot}}+T_{\mathrm{PM,tot}}}{T_{\mathrm{tot}}}=\frac{T_{\mathrm{BO,tot}}}{T_{\mathrm{tot}}}+\frac{T_{\mathrm{DP,tot}}}{T_{\mathrm{tot}}}+\frac{T_{\mathrm{PM,tot}}}{T_{\mathrm{tot}}}. \tag{16}$$

其中 ΔW_{tot} 和 W_{tot} 分别表示 R 轮次的总控制平面能量和总能量。使用表 5 中的分析结果，我们得到

$$\frac{T_{\mathrm{BO,tot}}}{T_{\mathrm{tot}}}\le 1.1\%,\quad \frac{T_{\mathrm{DP,tot}}}{T_{\mathrm{tot}}}\le 0.013\%,\quad \frac{T_{\mathrm{PM,tot}}}{T_{\mathrm{tot}}}\le 0.4\%.$$

因此，控制平面能量比例受到限制

$$\frac{\Delta W_{\mathrm{tot}}}{W_{\mathrm{tot}}}\le 1.1\%+0.013\%+0.4\%=1.513\%.$$

这表明，由于 PipeSD 的控制平面逻辑，边缘的额外能耗在我们的实验中最多占边缘总能耗的 1.513%，这是可以忽略的。

Table A.3. Average TPT (ms) under one-to-many cloud-edge deployment with fluctuating network conditions.

Clients	Vanilla	PipeSD	Reduction (%)
2	83.51	52.51	37.1
4	37.88	29.21	22.9
8	20.96	14.13	32.6

I. Extension to Multi-Edge Deployment

In practical cloud-edge collaborative inference scenarios, there may be multiple edge devices communicating with a single cloud server. Although PipeSD is described for a single edge device, it naturally generalizes to the multi-edge setting. Specifically, each edge device independently runs an instance of PipeSD, managing its own draft model, transmission controller, communication interface, environment monitor, and parameter updater. The cloud server maintains a single communication API and target model, handling requests from all edge devices.

To further validate the effectiveness of PipeSD in this setting, we additionally evaluate it under a one-to-many deployment with one cloud server and multiple edge clients. We consider a fluctuating-network setting in Scenario 4, where the communication bandwidth changes over time during inference. All clients are launched simultaneously and issue requests asynchronously under a full-load setting, i.e., each client starts a new inference task immediately after finishing the previous one.

Table A.3 shows that PipeSD consistently outperforms the vanilla cloud-edge speculative decoding baseline in the one-to-many setting. Specifically, PipeSD reduces TPT by 37.1%, 22.9%, and 32.6% for 2, 4, and 8 clients, respectively. These results indicate that PipeSD remains effective under concurrent multi-edge requests and fluctuating communication conditions. The gain mainly comes from two aspects: (i) the token-batch pipeline scheduling mechanism still helps overlap draft-token generation and transmission under dynamic network conditions, and (ii) the dual-threshold NAV triggering mechanism remains effective in determining verification triggering timing when multiple clients compete for cloud-side service resources. Overall, these results demonstrate that PipeSD can be extended from single-edge deployment to practical one-to-many cloud-edge serving scenarios.

J. Discussion of Tree-based Speculative Decoding

Tree-based speculative decoding (Miao et al., 2024) is an advanced speculative decoding approach that generates multiple draft token sequences in parallel, forming a tree of candidate continuations. Although this design can further reduce the frequency of NAV calls, it imposes substantial computational demands on edge devices and may result in the transmission of a large number of speculative tokens. As a consequence, tree-based speculative decoding can incur significant bandwidth overhead, particularly in bandwidth-constrained cloud-edge settings. Therefore, tree-based speculative decoding is less suitable for the cloud-edge deployment scenario considered in this work; nonetheless, PipeSD's techniques such as token-batch pipeline scheduling and dual-threshold NAV triggering can be adapted to enhance tree-based speculative decoding in future work.

Table A.3. 在网络状况波动的一对多云 - 边缘部署下的平均 TPT （毫秒）

Clients	Vanilla	PipeSD	Reduction (%)
2	83.51	52.51	37.1
4	37.88	29.21	22.9
8	20.96	14.13	32.6

s.

I. 扩展到多边缘部署

在实际的边云协同推理场景中，可能会有多个边缘设备与单个云服务器通信。虽然 PipeSD 是为单个边缘设备描述的，但它自然可以推广到多边缘设置。具体来说，每个边缘设备独立运行一个 PipeSD 实例，管理自己的草稿模型、传输控制器、通信接口、环境监控器和参数更新器。云服务器维护一个单一的通信 API 和目标模型，处理来自所有边缘设备的请求。

为了进一步验证 PipeSD 在此场景下的有效性，我们还在一对多部署环境中进行评估，其中包含一台云服务器和多个边缘客户端。我们在场景 4 中考虑了一个网络波动设置，其中通信带宽在推理过程中随时间变化。所有客户端同时启动，并在满负载设置下异步发出请求，即每个客户端在完成上一个推理任务后立即开始新的推理任务。

表 A.3 显示，在一对多设置中，PipeSD 一直优于原始的云 - 边推测解码基线。具体而言，PipeSD 将 2 、 4 和 8 个客户端的 TPT 分别降低了 37.1% 、 % 、 22.9% 和 % 、 32.6% 。这些结果表明，PipeSD 在并发多边请求和波动的通信条件下依然有效。收益主要来自两个方面：(i) token 批次流水线调度机制在动态网络条件下仍然有助于重叠草稿 token 的生成和传输，(ii) 双阈值 NAV 触发机制在确定多客户端时的验证触发时机仍然有效。争夺云端服务资源。总体而言，这些结果表明，PipeSD 可以从单边部署扩展到实际的一对多云边服务场景。

J. 基于树的推测解码讨论

基于树的推测解码 (Miao 等人 , 2024) 是一种先进的推测解码方法，它可以并行生成多个草稿 token 序列，形成候选续写的树状结构。尽管这种设计可以进一步减少 NAV 调用的频率，但它对边缘设备施加了巨大的计算负担，并且可能导致传输大量的推测 token。因此，基于树的推测解码可能会产生显著的带宽开销，特别是在带宽受限的云 - 边缘环境中。因此，基于树的推测解码对于本文考虑的云 - 边缘部署场景不太适合；尽管如此，PipeSD 的技术，例如 token 批处理流水线调度和双阈值 NAV 触发，可以在未来用于增强基于树的推测解码。工作 .